

数字货运系统部署手册

上海达牛信息技术有限公司

2023-11

版本	日期	作者	备注
V1.0	2023/11	张莉	初稿

目录

1	准备工作	5
1.1	应用运行环境介绍	5
1.2	采购阿里云服务	5
1.3	获取标准版的部署包或源码	5
1.4	用到的辅助工具	6
1.4.1	Navicat for MySQL	6
1.4.2	Robo 3T	6
1.4.3	Redis Desktop Manager	6
1.4.4	JDK	6
1.4.5	Nginx	6
1.4.6	Xshell	6
2	服务器配置	7
2.1	登录阿里云平台	7
2.1.1	云服务器 ECS 设置	11
2.1.2	云数据库 RDS 版设置	19
2.1.3	云数据库 MongoDB 版设置	28
2.1.4	云数据库 Redis 设置	42
2.1.5	SSL 证书申请	47
2.1.6	系统内核调优(参考)	51
2.1.7	应用目录创建	52
2.1.8	存储图片目录配置	53
2.1.9	Jdk 安装	53
2.1.10	Nginx 安装	54
2.1.11	Nacos 安装	59
2.1.12	Rabbitmq 安装	70
2.1.13	Canal 安装	70
3	安装部署	71
3.1	系统部署架构图	71
3.2	各应用简介	72

3.3	各应用间互访关系	73
3.4	脚本执行	73
3.4.1	脚本初始化	73
3.4.2	版本升级	74
3.5	jar 包部署	74
3.5.1	jar 启动脚本配置说明	74
3.5.2	web 部署	75
3.5.3	niux-gateway 部署	75
3.5.4	niux-activiti 部署	75
3.5.5	cms 部署	76
3.5.6	contract 部署	76
3.5.7	data 部署	77
3.5.8	dict 部署	77
3.5.9	eventbus 部署	78
3.5.10	file 部署	78
3.5.11	finance 部署	79
3.5.12	financing 部署	79
3.5.13	goods 部署	80
3.5.14	lbs 部署	80
3.5.15	openapi 部署	81
3.5.16	oms 部署	82
3.5.17	park 部署	82
3.5.18	payment 部署	83
3.5.19	planner 部署	83
3.5.20	policy 部署	84
3.5.21	supervise 部署	84
3.5.22	system 部署	85
3.5.23	tax 部署	85
3.5.24	asset 部署	86
3.5.25	tms 部署	86

3.5.26	trade 部署	87
3.5.27	vas 部署	87
3.5.28	wms 部署	88
3.5.29	report 部署	88
3.5.30	检查进程中未启动的 java 服务	89
3.5.31	检查重复的 java 服务	89
3.6	登录账号	89
4	小程序部署	89
4.1	申请账号	89
4.2	安装开发者工具	90
4.3	构建程序	90
4.4	修改配置文件	94
4.5	发布上线	95
	联系我们	95

1 准备工作

1.1 应用运行环境介绍

JDK: JDK8.0 以上

Mysql: Mysql5.7

MongoDB: MongoDB4.2

Redis: Redis5

Nginx 代理服务器: nginx-1.25 以上

Nacos 配置管理和注册中心: 2.2.3 以上

服务器 ip: 以 192.168.0.1 为例

1.2 采购阿里云服务

部署本系统需要采购云服务器 ECS、弹性公网 IP、云数据 RDS 版、云数据库 Mongo 版以及云数据库 Redis 版本产品。购买配置参考如下:

服务器名称	个数	配置	系统要求	用途
云服务器 ECS	1	4 核 CPU、32GB 内存、100G 系统盘、500G 数据盘	64 位 centos7.4	Web 服务器
云数据库 MongoDB 版	1	1 核 CPU、2GB 内存、50GB 存储	mongoDB 4.2 副本集	位置数据库
云数据库 RDS 版	1	8 核 CPU、16GB 内存、100GB 存储 MySQL5.7	mysql5.7 双机高可用版本	业务数据库
云数据 Redis 版	1	1G 主从版	1G 主从版	缓存服务器
弹性公网 IP	1	5M 带宽		
备注: 以上服务器必须在同一地域内。				

1.3 获取标准版的部署包或源码

1、源码获取方式

下载标准版应用程序包

Git: http://gitlab.niuiinfo.com/developement/open_tms

账号和密码请联系达牛商务。

2、jar 包部署方式

请联系达牛商务

1.4 用到的辅助工具

1.4.1 Navicat for MySQL

Navicat for MySQL 是管理和开发 MySQL 或 MariaDB 的理想解决方案。它是一套单一的应用程序，能同时连接 MySQL 和 MariaDB 数据库，并与 Amazon RDS、Amazon Aurora、Oracle Cloud、Microsoft Azure、阿里云、腾讯云和华为云等云数据库兼容。这套全面的前端工具为数据库管理、开发和维护提供了一款直观而强大的图形界面。可以从 <https://www.navicat.com.cn/download/navicat-for-mysql> 链接下载安装。

1.4.2 Robo 3T

Robo 3T 是一个支持 windows、Mac、Linux 三个平台的 mongo 图形化客户端。可以从官网 <https://robomongo.org/download> 下载安装。

1.4.3 Redis Desktop Manager

RedisDesktopManager 是一个支持 windows 平台的 redis 图形化客户端，可以从 <https://github.com/uglide/RedisDesktopManager/releases/tag/0.8.8> 下载。

1.4.4 JDK

JDK 是 Java 语言的软件开发工具包，主要用于移动设备、嵌入式设备上的 java 应用程序。JDK 是整个 java 开发的核心，它包含了 JAVA 的运行环境，JAVA 工具和 JAVA 基础的类库。可以从官网 <http://www.oracle.com/technetwork/java/javase/downloads/> 下载安装。

1.4.5 Nginx

Nginx (engine x) 是一个高性能的 HTTP 和反向代理 web 服务器，下载说明参考 http://nginx.org/en/linux_packages.html#RHEL-CentOS。

1.4.6 Xshell

安全终端仿真器 Xshell 是一个运行于 MS Windows 平台上强大的 SSH, TELNET 和 RLOGIN 终端仿真器。用户可以从 Windows PC 上安全的访问 Unix/Linux 主机。下载说明参考 <https://www.netsarang.com/zh/xshell/>。

2 服务器配置

以阿里云平台为例

2.1 登录阿里云平台

打开登录网址 <https://account.aliyun.com/>，输入用户名密码进行登录



登录后选择左上角图标->产品与服务->云服务器 ECS->实例与镜像->实例->选择区域







账号全部资源

华北2 (北京)

云服务器 ECS / 实例

例

创建实例

...

Q 自动识别

实例 ID / 名称

亚太 - 中国

华东1 (杭州)

华东2 (上海)

华东5 (南京-本地地域)

华北1 (青岛)

华北2 (北京)

华北3 (张家口)

华北5 (呼和浩特)

华北6 (乌兰察布)

华南1 (深圳)

华南2 (河源)

华南3 (广州)

西南1 (成都)

中国香港

华东6 (福州-本地地域)

华中1 (武汉-本地地域)

13

欧洲与美洲

德国 (法兰克福)

英国 (伦敦)

美国 (硅谷)

美国 (弗吉尼亚)

中东

阿联酋 (迪拜)

亚太 - 其他

日本 (东京)

韩国 (首尔)

新加坡

澳大利亚 (悉尼)

马来西亚 (吉隆坡)

印度尼西亚 (雅加达)

菲律宾 (马尼拉)

2.1.1 云服务器 ECS 设置

2.1.1.1 查看 ECS 实例

点击菜单“云服务器 ECS”

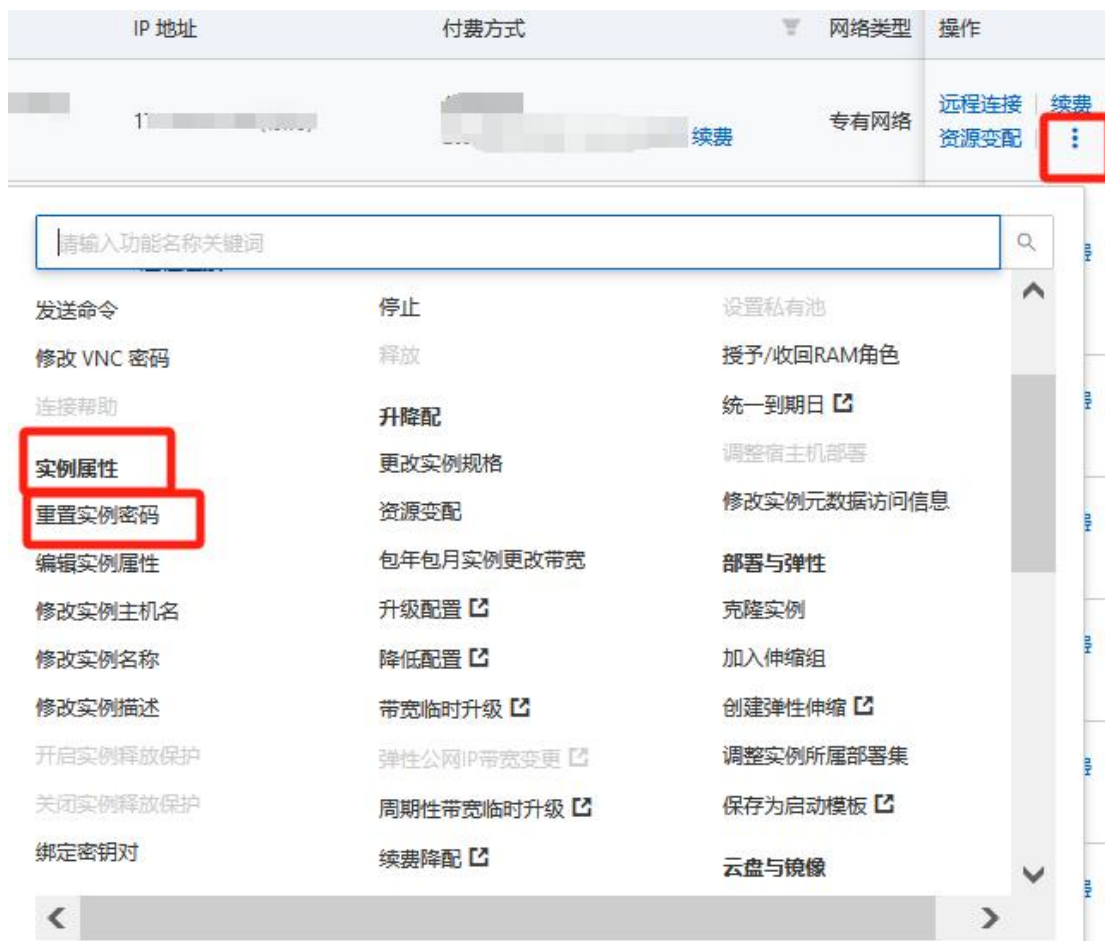


然后点击实例与镜像下的子菜单“实例”，查看 ECS 实例信息



2.1.1.2 设置密码

设置密码，选择 ECS 实例->选择最右边 (更多)->实例属性->重置实例密码



输入密码后提交完成

重置实例密码

重置密码的方式

☒ 在线重置密码

密码修改后无需重启即可生效；支持修改管理员账号及非管理员账号的密码；
 通过云助手进行密码本文的加密传输，任何人都无法解密该内容，确保密码安全。

☐ 离线重置密码

仅支持对您在创建实例时选择的登录名修改密码，密码修改后需要重启实例方可生效。

如果实例已绑定密钥对，重置密码后系统将通过新密码进行登录。

您所选的 1 个实例 将执行重置实例密码操作，您是否确认操作？

登录名

root

* 新密码

8-30个字符且只能包含以下限制条件中的字符

a小写字母

A大写字母

1数字

只可包含特殊字符`~!@#\$%^&*~+=|{};:'<>.,?/

密码完成度

* 确认密码

配置SSH密码登录策略

☒ 开启
 ☐ 保持原有设置

确定

取消

2.1.1.3 重启云服务器 ECS

重启云服务器 ECS，选择 ECS 实例->选择最右边 (更多)->实例状态->重启



2.1.1.4 安全组设置

设置安全组，选择 ECS 实例->选择实例 id 打开->安全组



点击“管理规则”



点击“快速创建规则”



填写如下并保存。

特别注意，除 https/http 端口外，其他端口从安全角度考虑不允许对 0.0.0.0/0 开放；

将此处示例 111.111.111.111 修改为自己的出口 IP，出口 IP 如无特殊配置，可以通过浏览器访问 <https://www.ip138.com> 或者 [curl ifconfig.me](https://curl.ifconfig.me) 获取。

快速添加
×

授权策略
* 授权对象:

允许
▼

0.0.0.0/0 ×

* 端口范围:

☐ SSH (22)
☒ HTTPS (443)
☐ MySQL (3306)
☐ Redis (6379)

☐ telnet (23)
☐ MS SQL (1433)
☐ RDP (3389)
☐ 全部 (1/65535)

☒ HTTP (80)
☐ Oracle (1521)
☐ PostgreSQL (5432)

方向	授权策略	优先级	协议类型	端口范围	授权对象	描述
入方向	允许	1	自定义 TCP	目的: 80/80 443/443	源: 0.0.0.0/0	

确定
取消

快速添加 X

授权策略 * 授权对象:

允许 111.111.111.111 X

* 端口范围:

☒ SSH (22)
☒ HTTPS (443)
☒ MySQL (3306)
☒ Redis (6379)

☒ telnet (23)
☐ MS SQL (1433)
☐ RDP (3389)
☐ 全部 (1/65535)

☒ HTTP (80)
☐ Oracle (1521)
☐ PostgreSQL (5432)

方向	授权策略	优先级	协议类型	端口范围	授权对象	描述
入方向	允许	1	自定义 TCP	目的: 22/22 3306/3306 6379/6379 443/443 80/80 23/23	源: 111.111.111.111	

确定 取消

点击“手动添加”

访问规则 导入安全组规则 导出 健康检查

入方向 出方向

手动添加 快速添加 输入端口或者授权对象进行搜索 不合并

授权策略	优先级	协议类型	端口范围	授权对象	描述	创建时间	操作
------	-----	------	------	------	----	------	----

填写如下并保存。

特别注意，除 https/http 端口外，其他端口从安全角度考虑不允许对 0.0.0.0/0 开

放；

将此处示例 111.111.111.111 修改为自己的出口 IP，出口 IP 如无特殊配置，可以通过浏览器访问 <https://www.ip138.com> 或者 [curl ifconfig.me](https://curl.ifconfig.me) 获取。

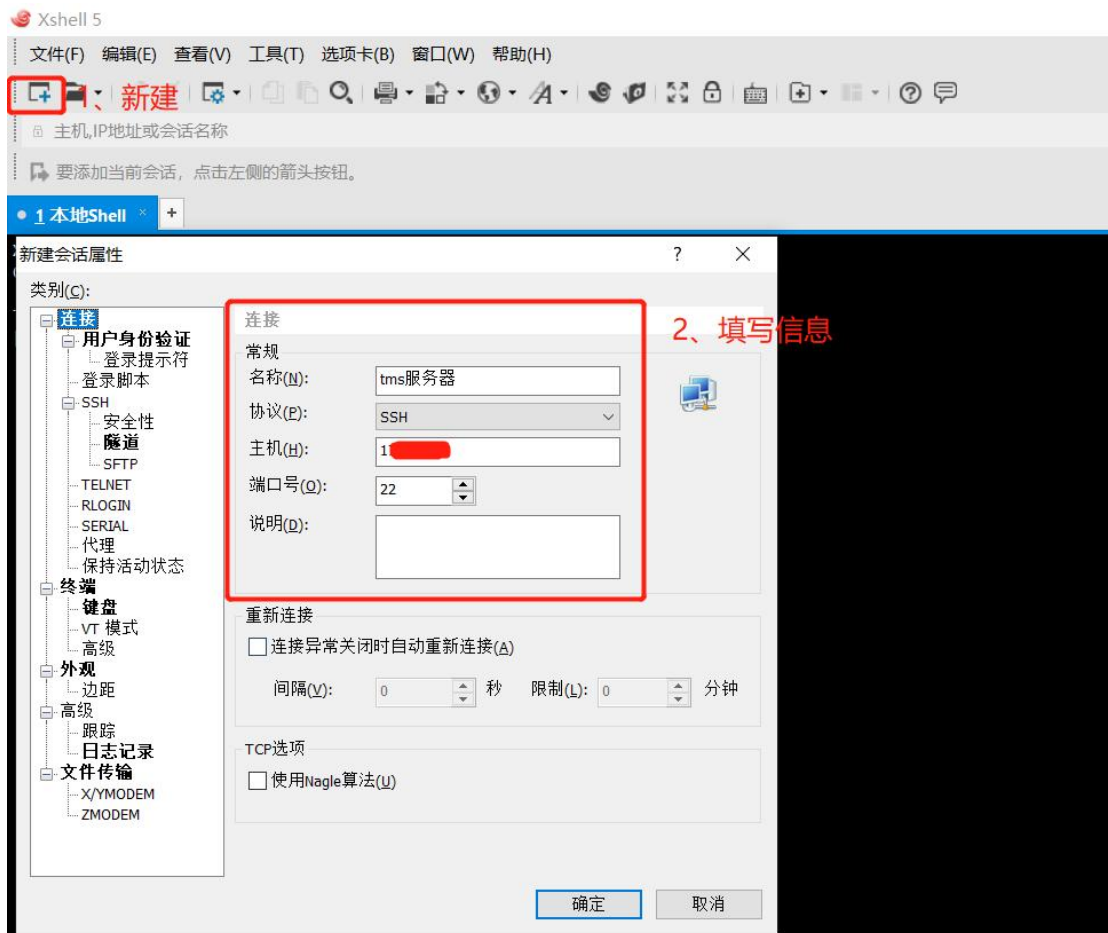
入方向 出方向

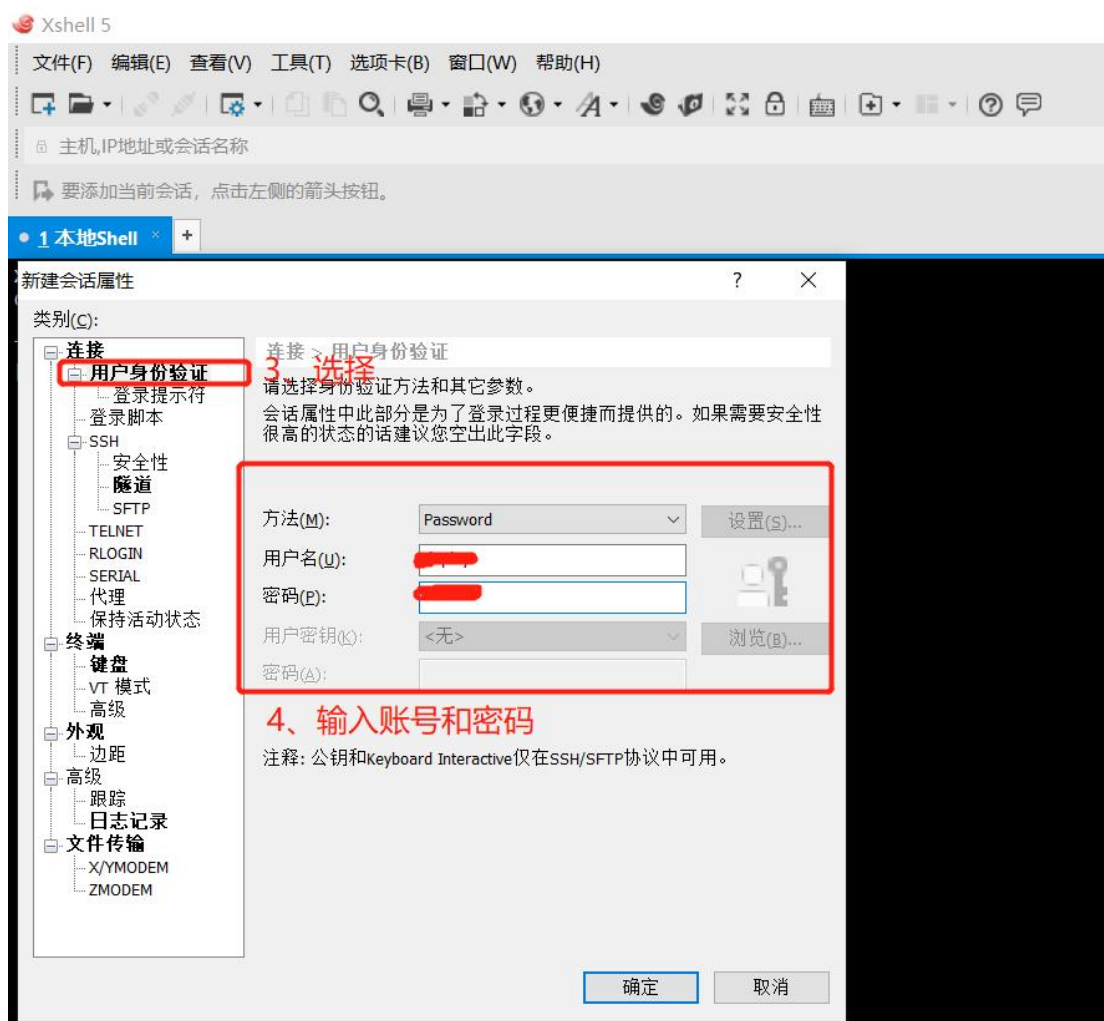
手动添加 快速添加 输入端口或者授权对象进行搜索 不合并

授权策略	优先级	协议类型	端口范围	授权对象	描述	操作
允许	1	自定义 TCP	目的: 22/22/2017	源: 111.111.111.111	职场	保存 预览 删除

2.1.1.5 连接 ECS

通过 Xshell 远程连接 ECS

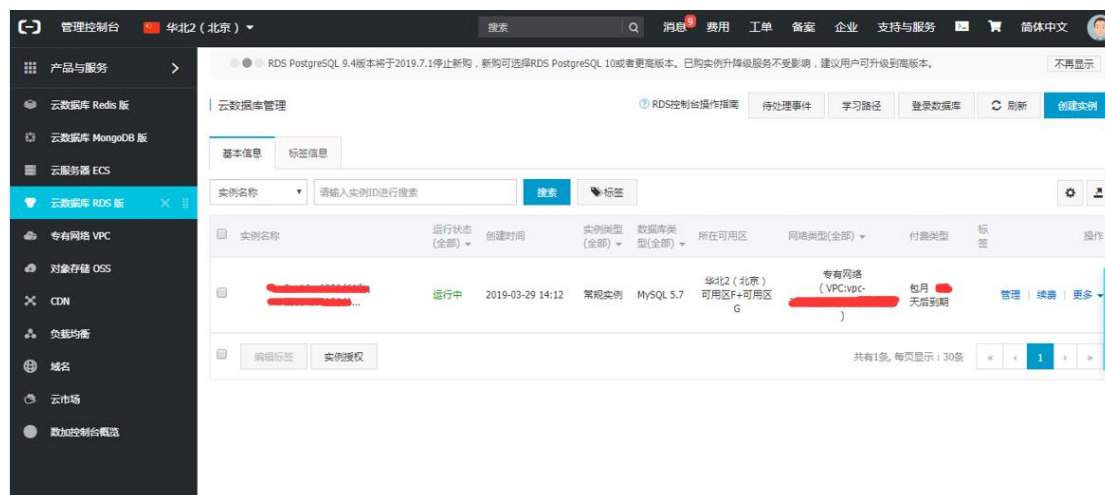




2.1.2 云数据库 RDS 版设置

2.1.2.1 查看“云数据库 RDS 版”

点击左边菜单“云数据库 RDS 版”如下图



2.1.2.2 设置白名单

点击“管理”



设置白名单



添加白名单分组



输入设置参数，然后保存

添加白名单分组

网络隔离模式: ☒ 专有网络 ☐ 经典网络 及 外网地址
 请先切换高安全白名单模式, 再进行网络隔离配置。(当前不可配置)

分组名称*: intranet_rds_ip_list

组内白名单*: 172.17.0.0/16

加载ECS内网IP 还可添加999个白名单

指定IP地址: 192.168.0.1 允许192.168.0.1的IP地址访问RDS
 指定IP段: 192.168.0.0/24 允许从192.168.0.1到192.168.0.255的IP地址访问RDS
 多个IP设置, 用英文逗号隔开, 如192.168.0.1,192.168.0.0/24
[如何定位本地IP](#)

新白名单将于1分钟后生效

确定 取消

保存后如下图

管理控制台

数据安全

白名单设置 SQL审计 SSL

网络隔离模式: 通用白名单模式。以下白名单不区分经典网络及专有网络。

切换高安全白名单模式 (推荐) + 添加白名单分组

白名单名称	白名单地址	操作
default	127.0.0.1	修改 清空
intranet_rds_ip_list	172.17.0.0/16	修改 删除

注: RDS IP白名单设计支持IP段如X.X.X.X/X的配置, 设置为127.0.0.1代表禁止所有地址访问。 [白名单设置说明](#)

安全组

安全组

+ 添加安全组 清空

点击“基本信息”, 可以查看 RDS 内网信息



2.1.2.3 查看连接信息

rm-bcdefgh.mysql.rds.aliyuncs.com:3306

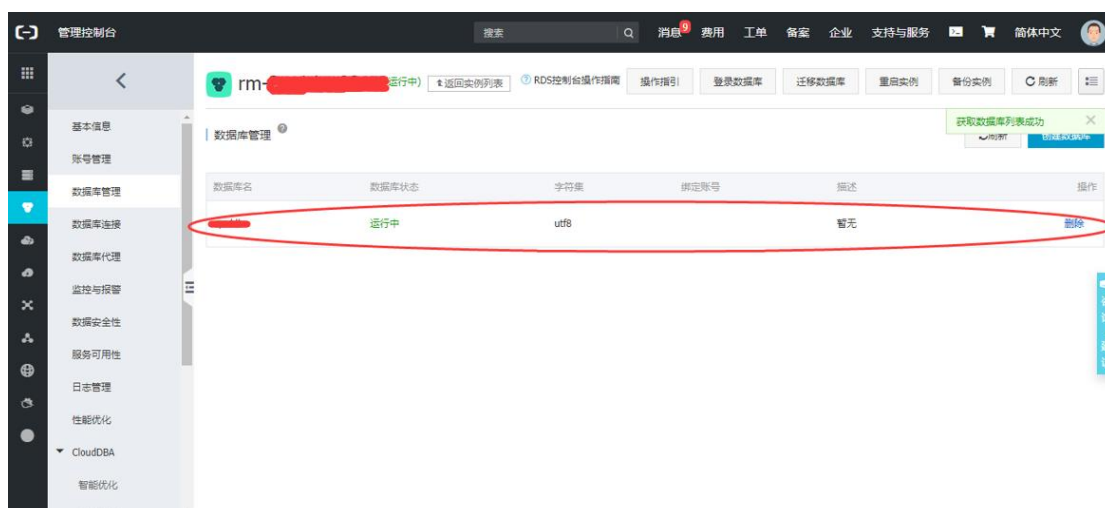
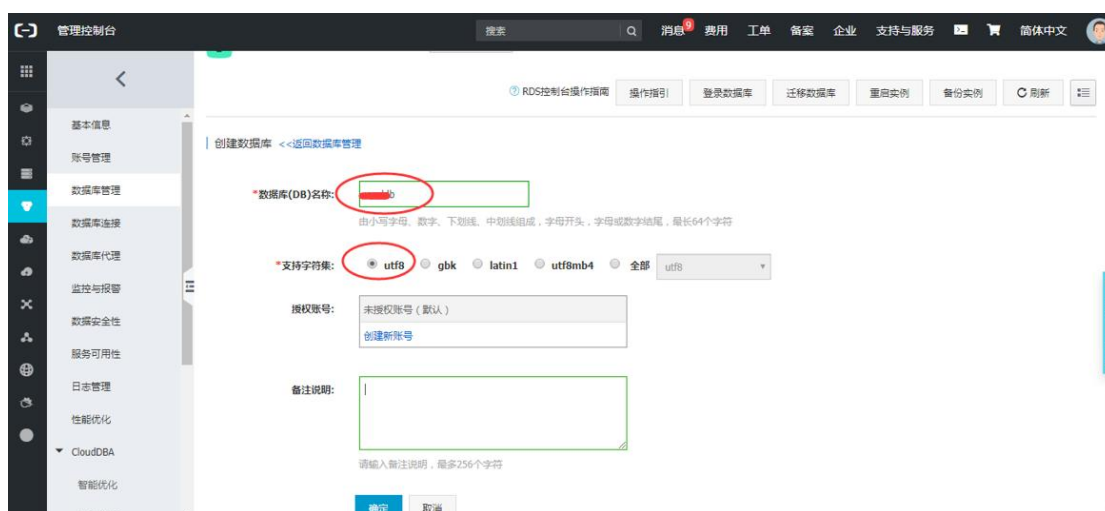


2.1.2.4 创建数据库

打开子菜单“数据库管理”，点击“创建数据库”



输入数据库名，然后保存



2.1.2.5 创建账号

打开子菜单“账号管理”，然后点击“创建账号”



输入账号信息，然后保存

*账号类型: ☐ 高权限账号 ☒ 普通账号

授权数据库: 未授权数据库 (暂无数据) | 已授权数据库 (权限: 全部 读写)

密码: (大写、小写、数字、特殊字符占三种, 长度为8-32位; 特殊字符为!@#\$%^&*()_+)

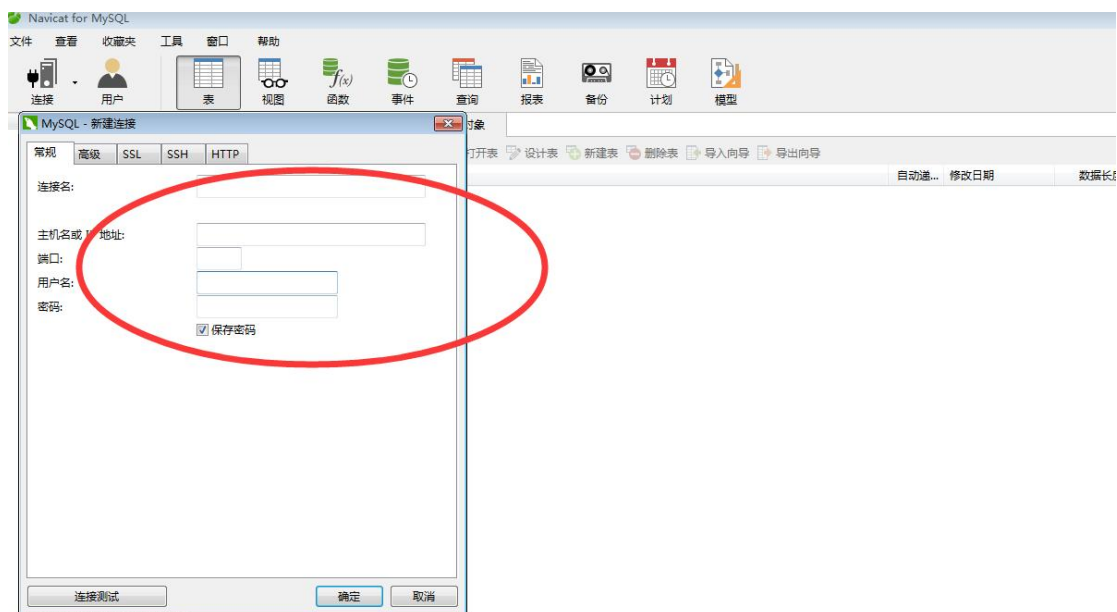
确认密码:

备注说明:

智能顾问: 智能诊断, 秒级解 | 聆听·建议反馈 | 阿里云不是完美的

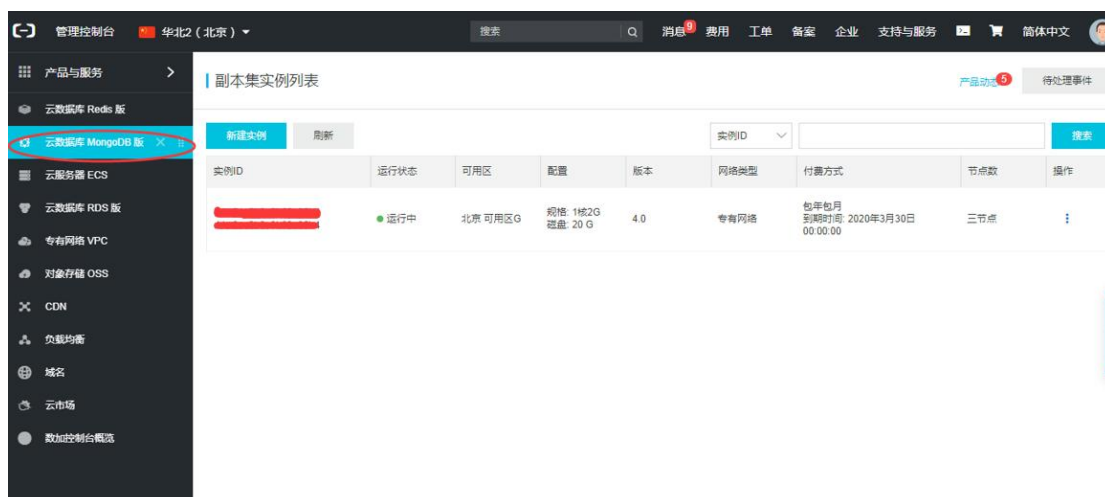


2.1.2.6 连接 mysql

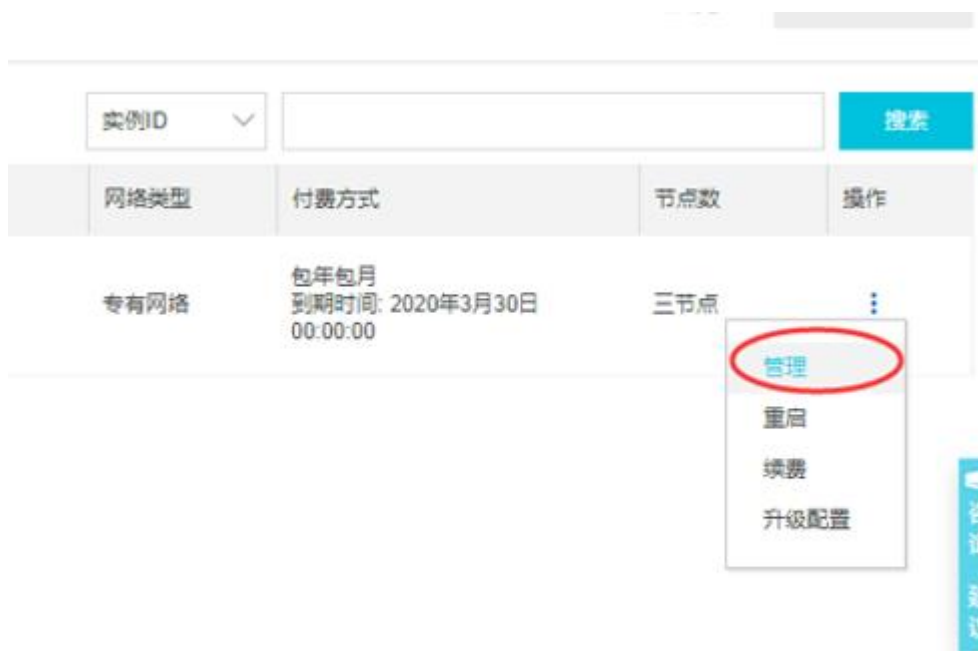


2.1.3 云数据库 MongoDB 版设置

2.1.3.1 查看实例信息



2.1.3.2 设置白名单



The screenshot shows the 'Management Console' (管理控制台) interface. On the left is a sidebar with navigation options: 账号管理, 数据库连接, 备份与恢复, 监控信息, 报警规则, 参数设置, 数据安全, 白名单设置, 审计日志, SSL, and 日志管理. The main area displays instance details for '规格详情 1核2G'. Below this, the '连接信息 (Connection String URI)' section is highlighted with a red circle around the text '设置白名单后才显示连接地址'. The '节点列表' (Node List) table shows two nodes: Primary and Secondary, both with redacted IDs and the same instruction '设置白名单后才显示连接地址'.

角色	角色ID	域名信息
Primary	[Redacted]	设置白名单后才显示连接地址
Secondary	[Redacted]	设置白名单后才显示连接地址

点击“添加白名单分组”

The screenshot shows the 'Management Console' (管理控制台) interface. The '白名单设置' (White List Settings) option in the sidebar is highlighted. The main area shows the '实例 dds-[Redacted]' (Instance dds-[Redacted]) status as '运行中' (Running). A blue button labeled '添加白名单分组' (Add White List Group) is circled in red. Below the button is a table with two columns: '分组名' (Group Name) and '允许访问IP名单' (Allowed Access IP List). The table contains one entry: 'default' with the IP '127.0.0.1'. Below the table, it says '您已添加1个IP, 还能添加999个' (You have added 1 IP, you can still add 999 more).

分组名	允许访问IP名单
default	127.0.0.1

输入白名单内网 ip 信息后进行确认

添加白名单分组

① 0.0.0.0/0和空代表不设IP访问的限制，数据库将会有高安全风险。建议仅将您的WEB服务器外网IP/IP段设为可访问权限

* 分组名

intranet_mongo_ip_list

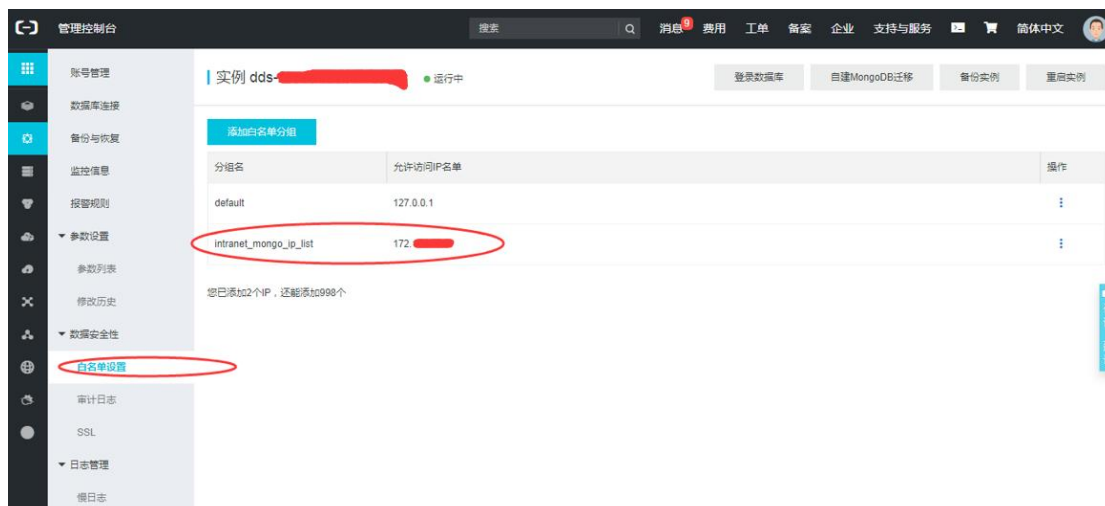
22/32

* 允许访问IP名单

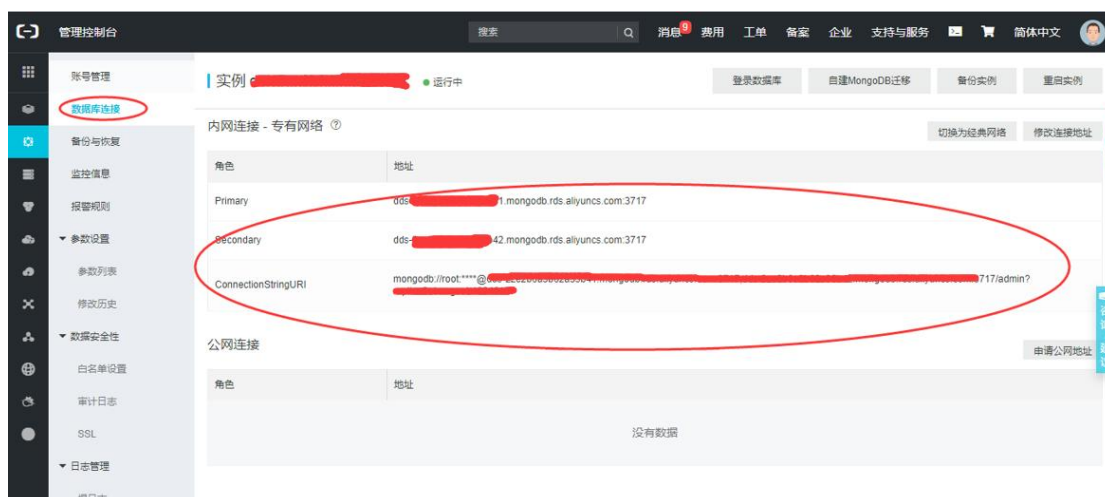
172. [REDACTED]

确定

取消

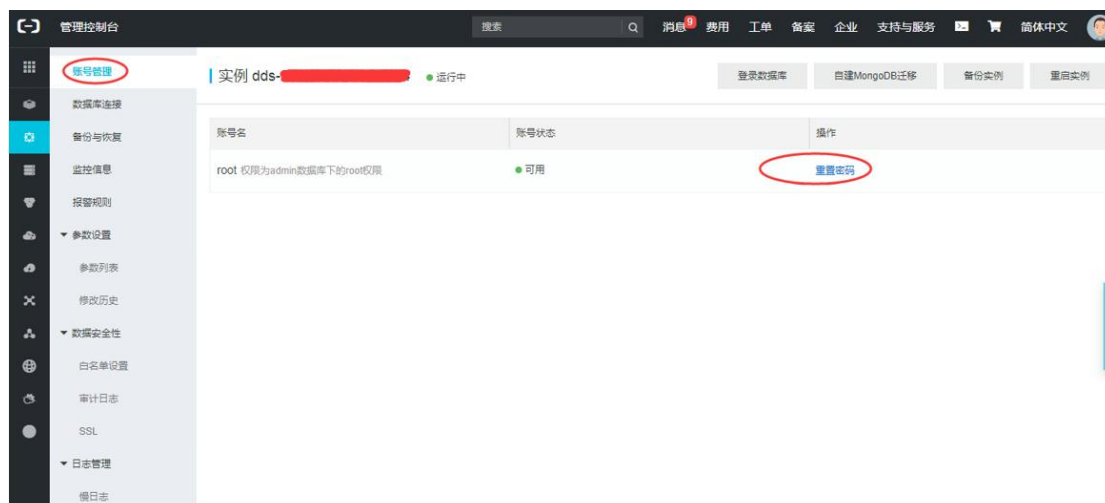


2.1.3.3 查看数据库连接

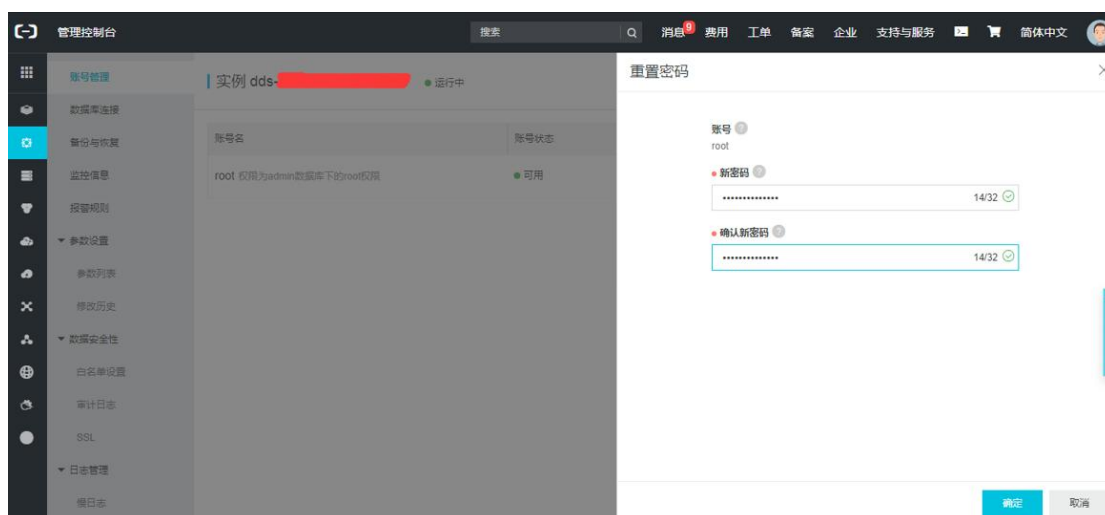


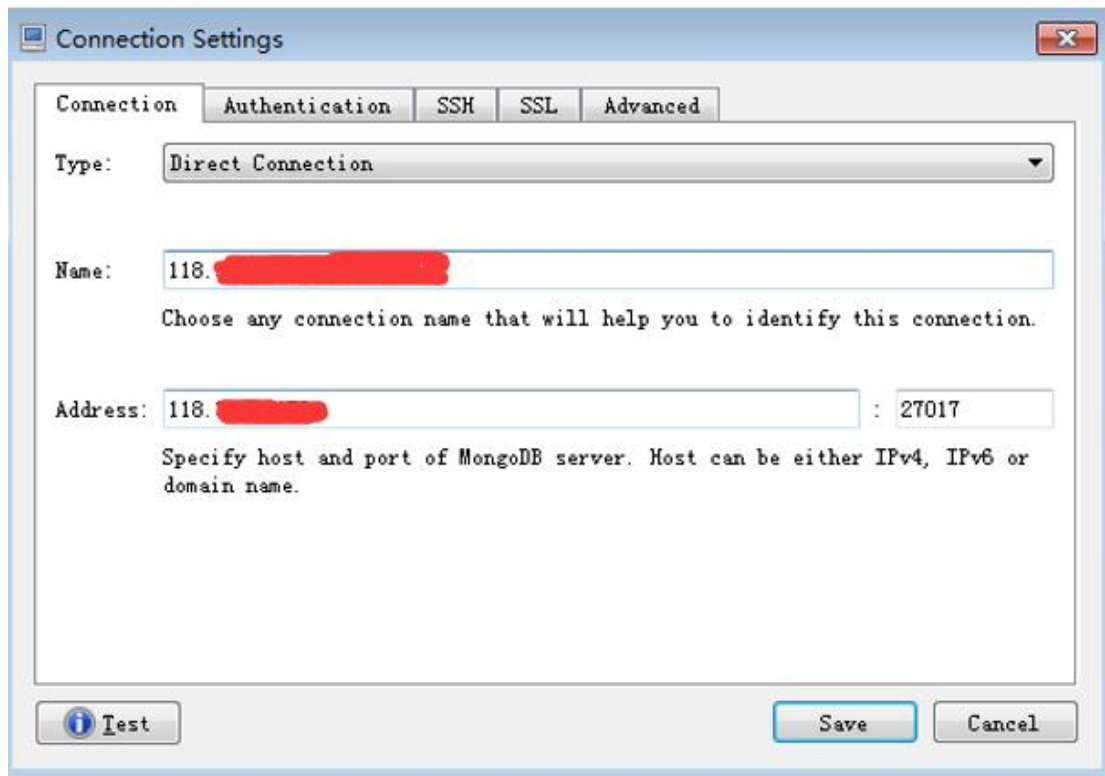
mongodb://root:****@dds-abcdefg41.mongodb.rds.aliyuncs.com:3717,dds-
abcdefg42.mongodb.rds.aliyuncs.com:3717/admin?replicaSet=mgset-123456

2.1.3.4 Dds 实例重置 root 密码



输入密码进行确认





Connection Settings

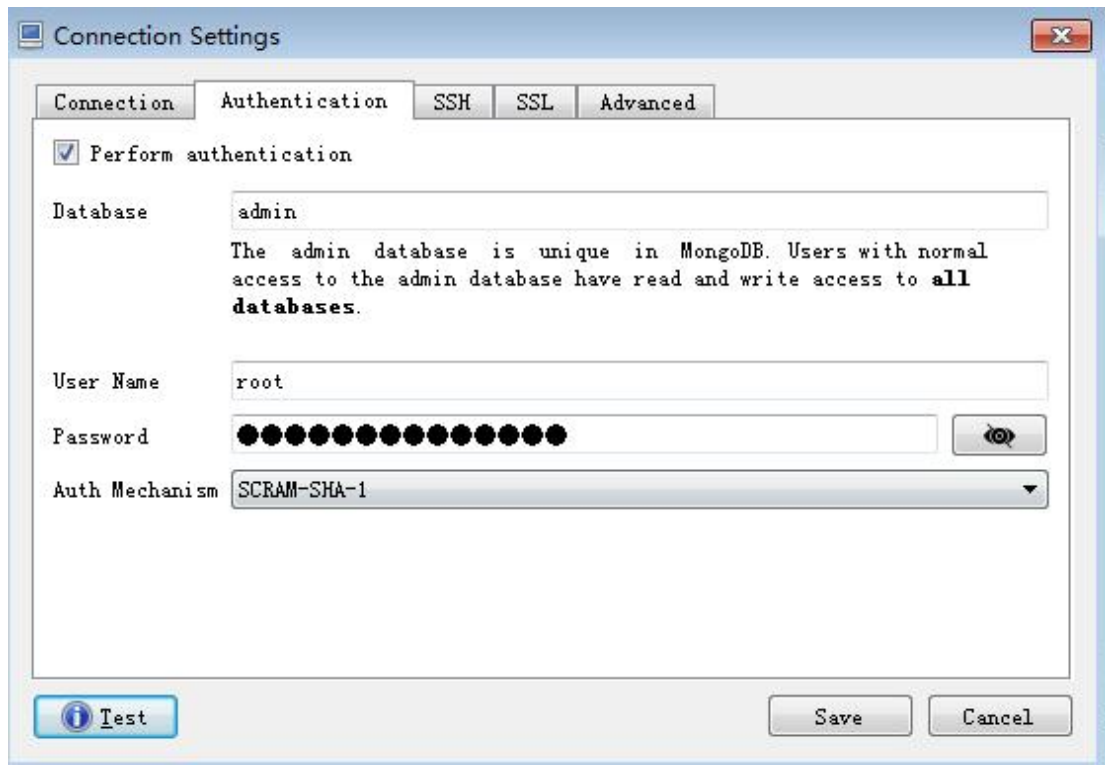
Connection | Authentication | SSH | SSL | Advanced

Type: Direct Connection

Name: 118. [REDACTED]
Choose any connection name that will help you to identify this connection.

Address: 118. [REDACTED] : 27017
Specify host and port of MongoDB server. Host can be either IPv4, IPv6 or domain name.

Test Save Cancel



Connection Settings

Connection | Authentication | SSH | SSL | Advanced

☒ Perform authentication

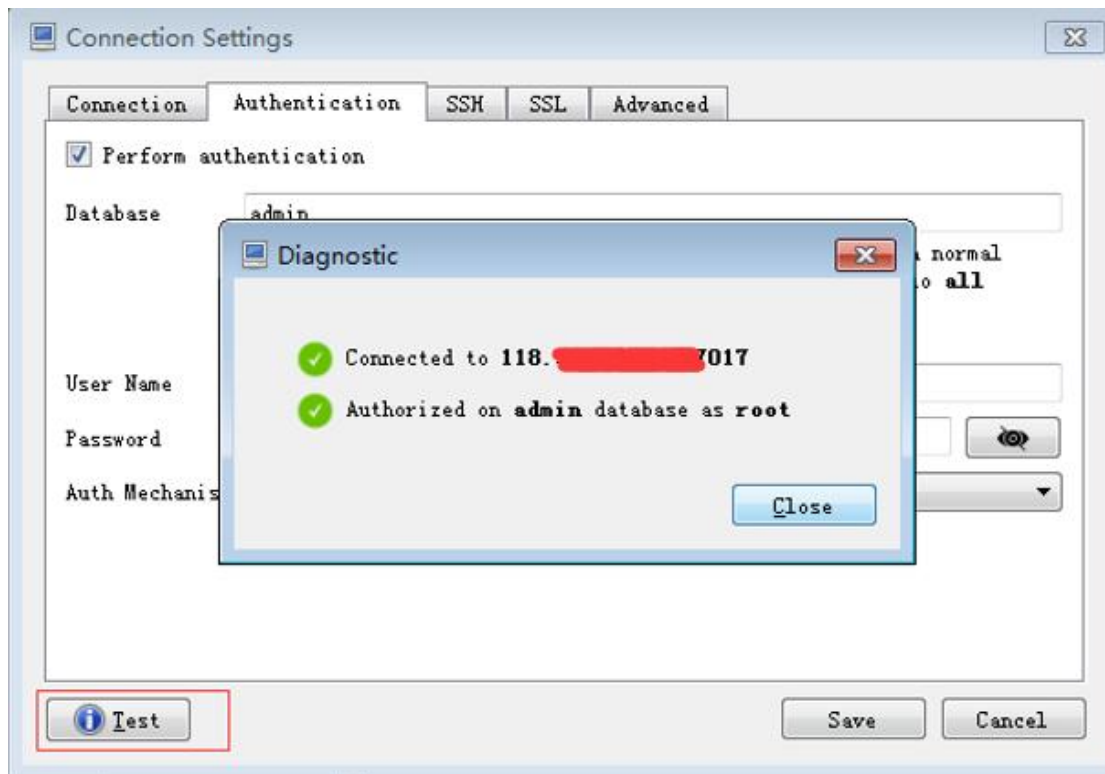
Database: admin
The admin database is unique in MongoDB. Users with normal access to the admin database have read and write access to **all** databases.

User Name: root

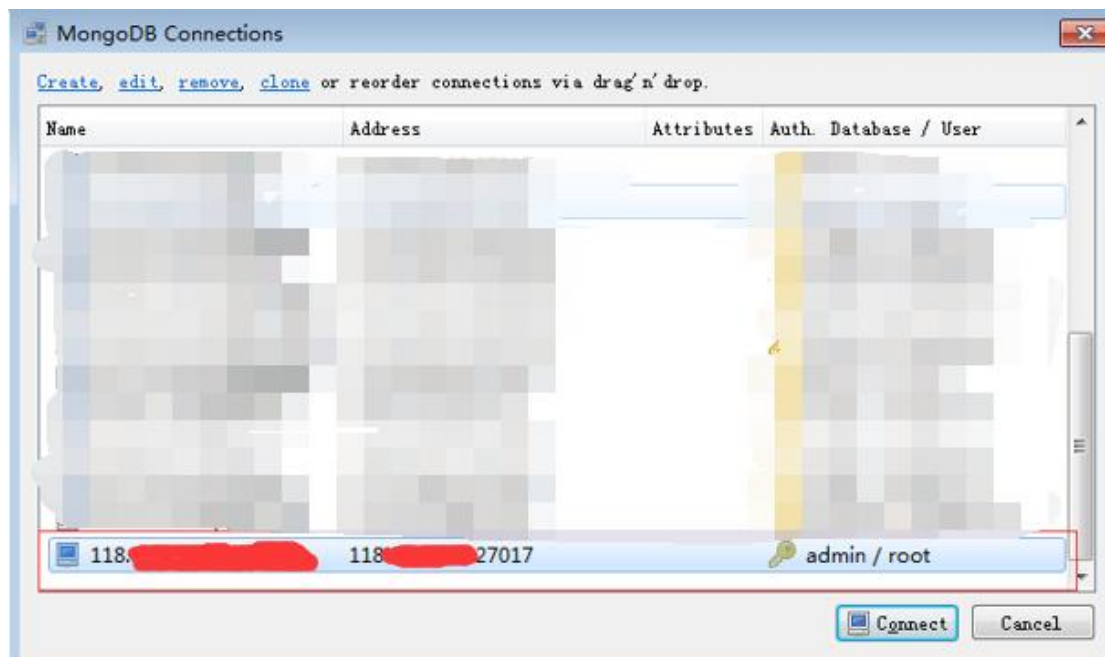
Password: [REDACTED] [Eye Icon]

Auth Mechanism: SCRAM-SHA-1

Test Save Cancel



这个又是什么工具？



新增数据库

use tmsdb

db.createUser(

{

user: "tmsdbuser",

```

pwd: "tmspassword",

roles: [

  { role: "readWrite", db: "tmsdb" }

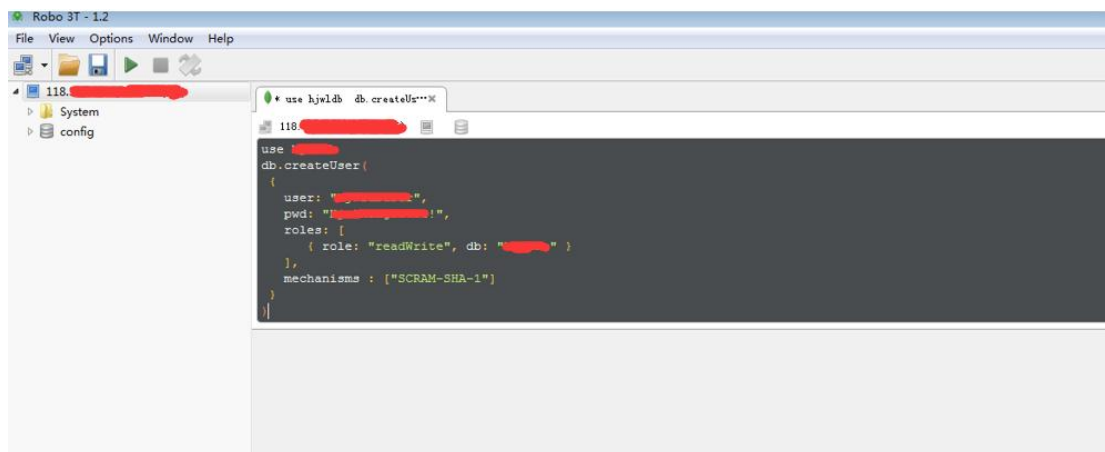
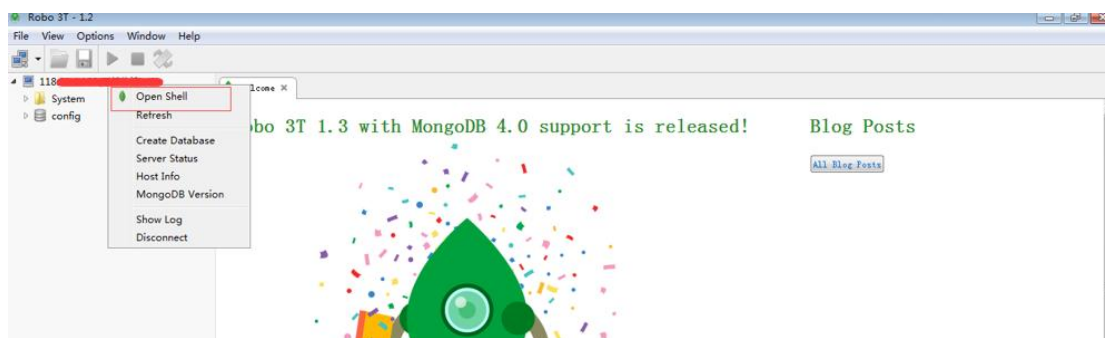
],

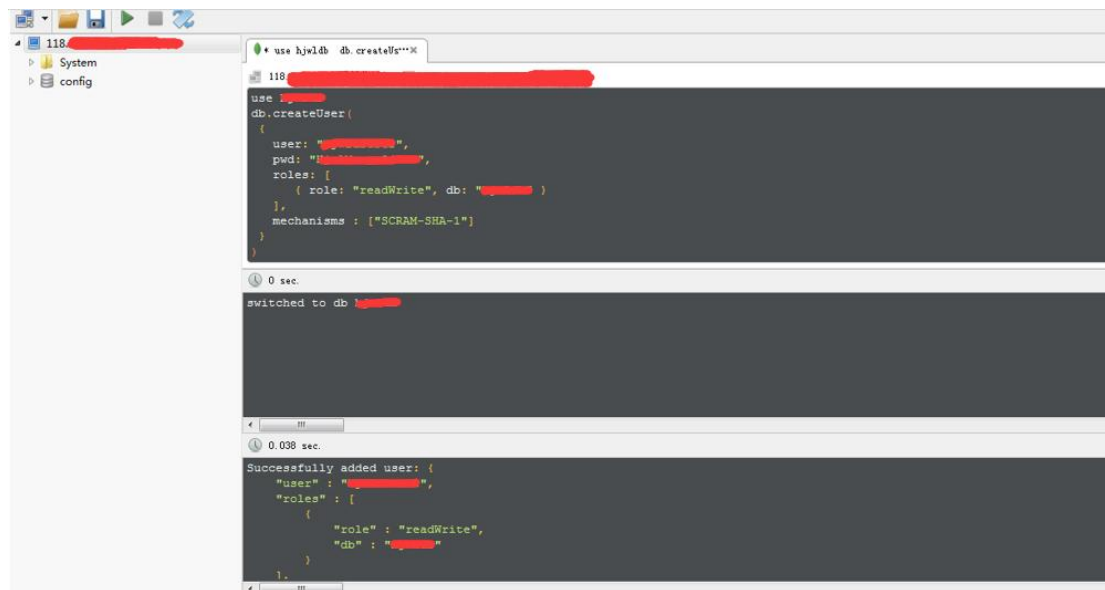
mechanisms : ["SCRAM-SHA-1"]

}

)

```





The screenshot shows a MongoDB shell session. The user has entered the command `use hjiadb db.createUser()` and the following JSON document:

```

{
  user: "hjiadb",
  pwd: "hjiadb",
  roles: [
    { role: "readWrite", db: "hjiadb" }
  ],
  mechanisms: ["SCRAM-SHA-1"]
}

```

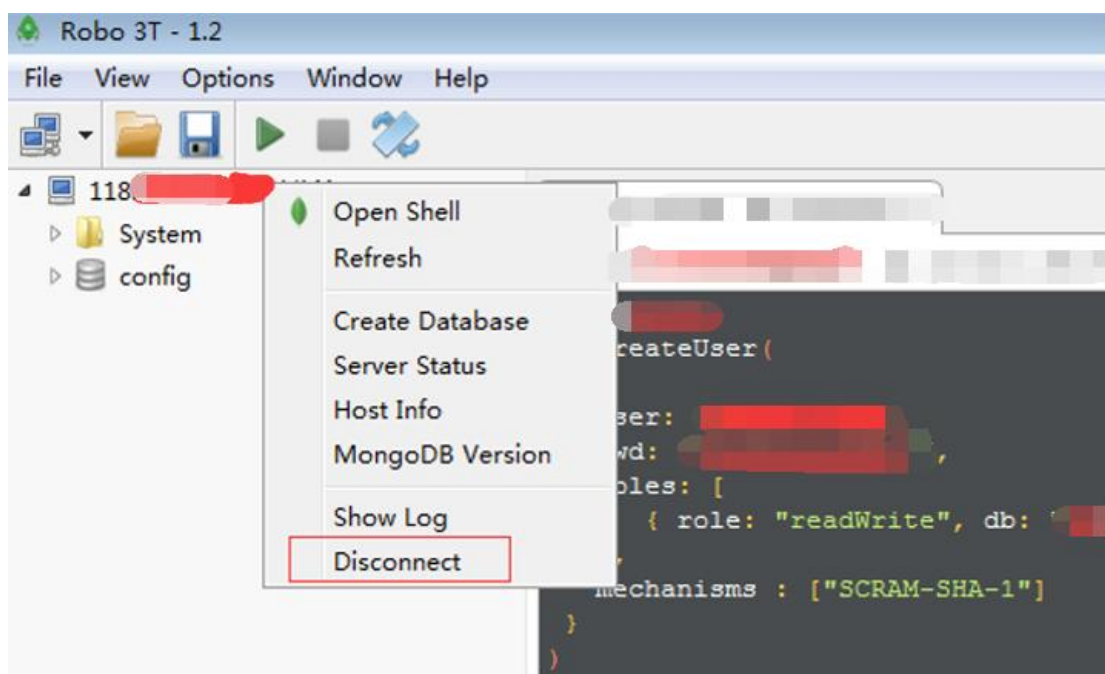
The shell output shows the command was executed successfully, switching to the `hjiadb` database and confirming the user was added.

```

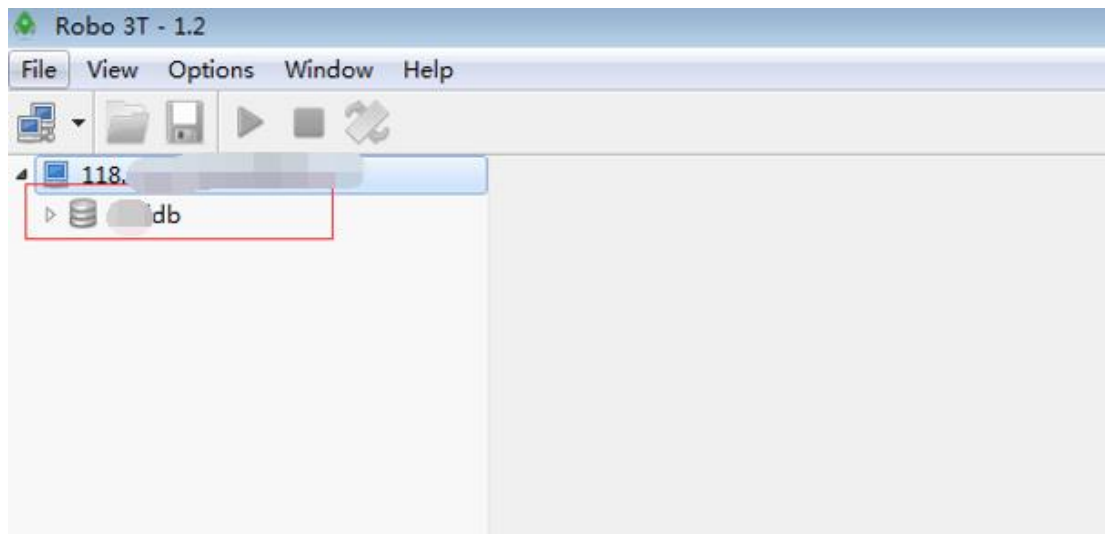
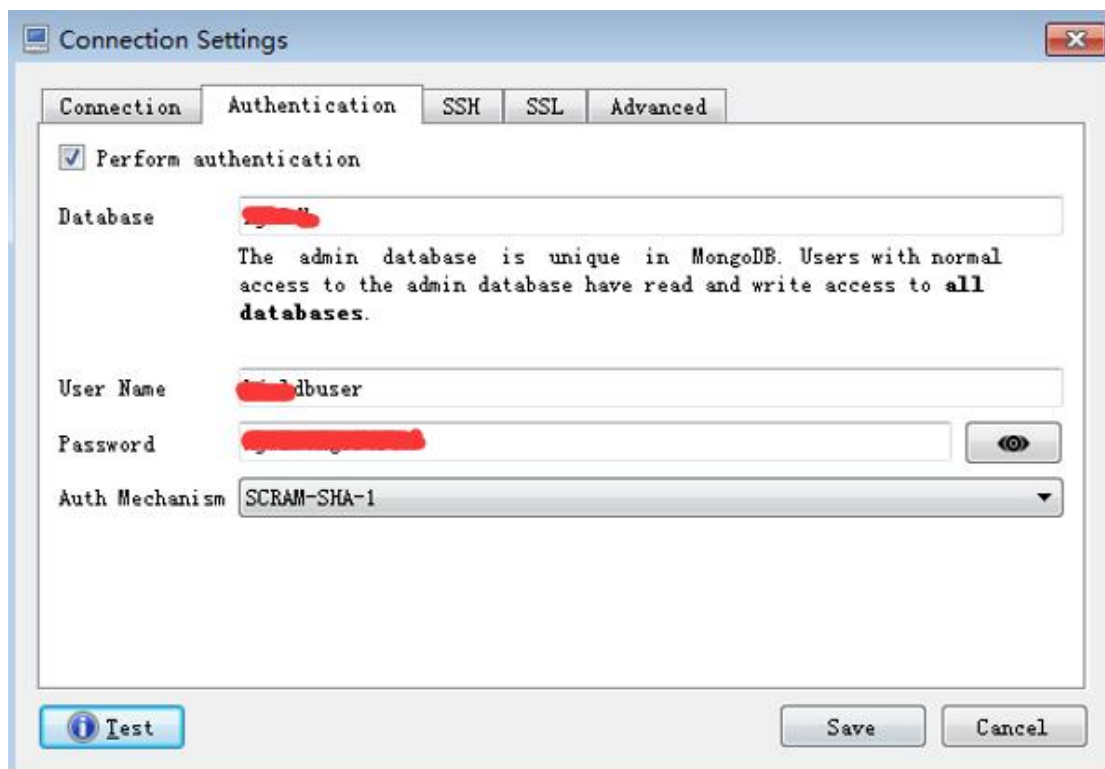
switched to db hjiadb

Successfully added user: {
  "user" : "hjiadb",
  "roles" : [
    {
      "role" : "readWrite",
      "db" : "hjiadb"
    }
  ]
}

```

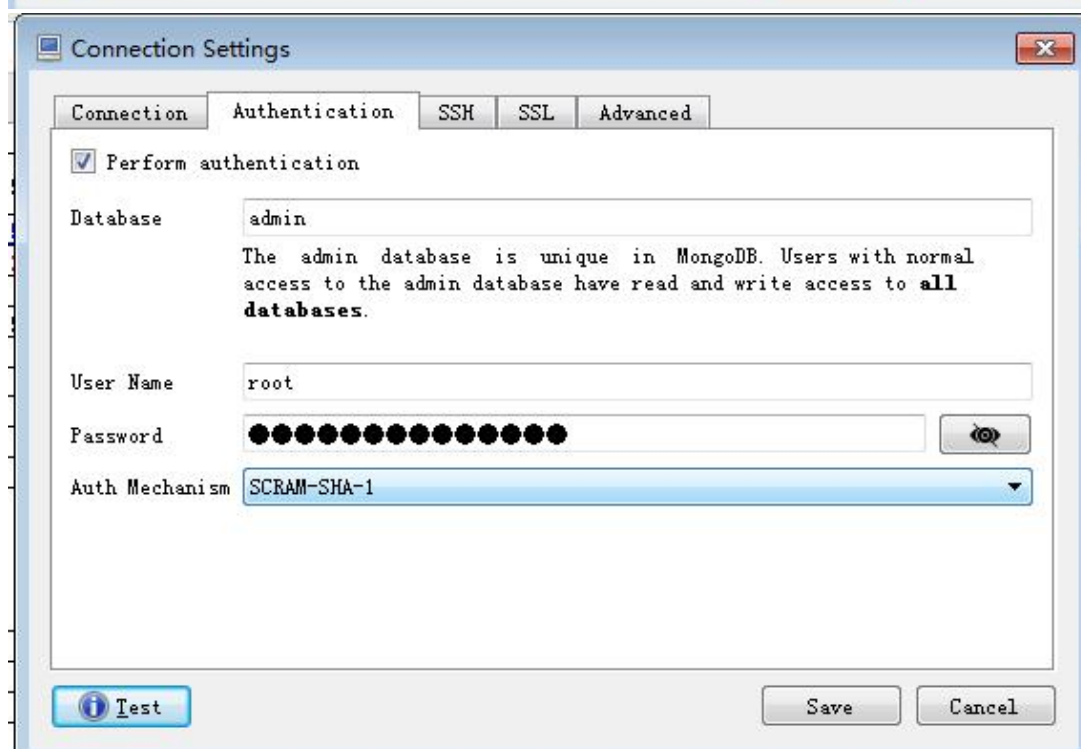
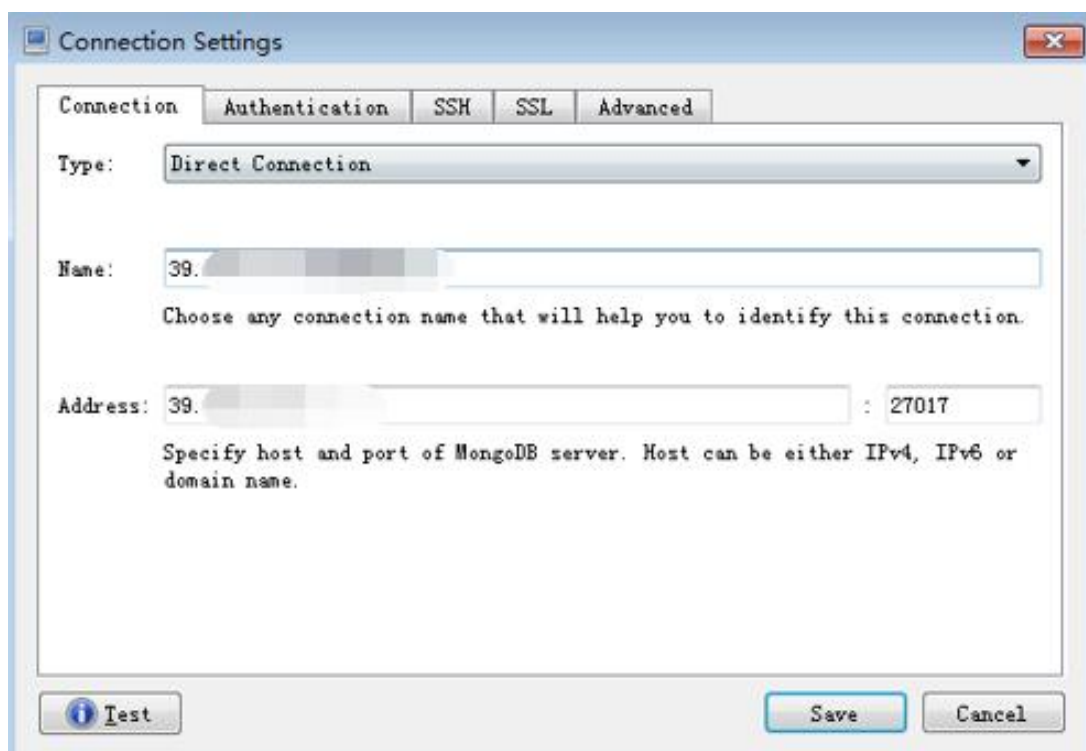


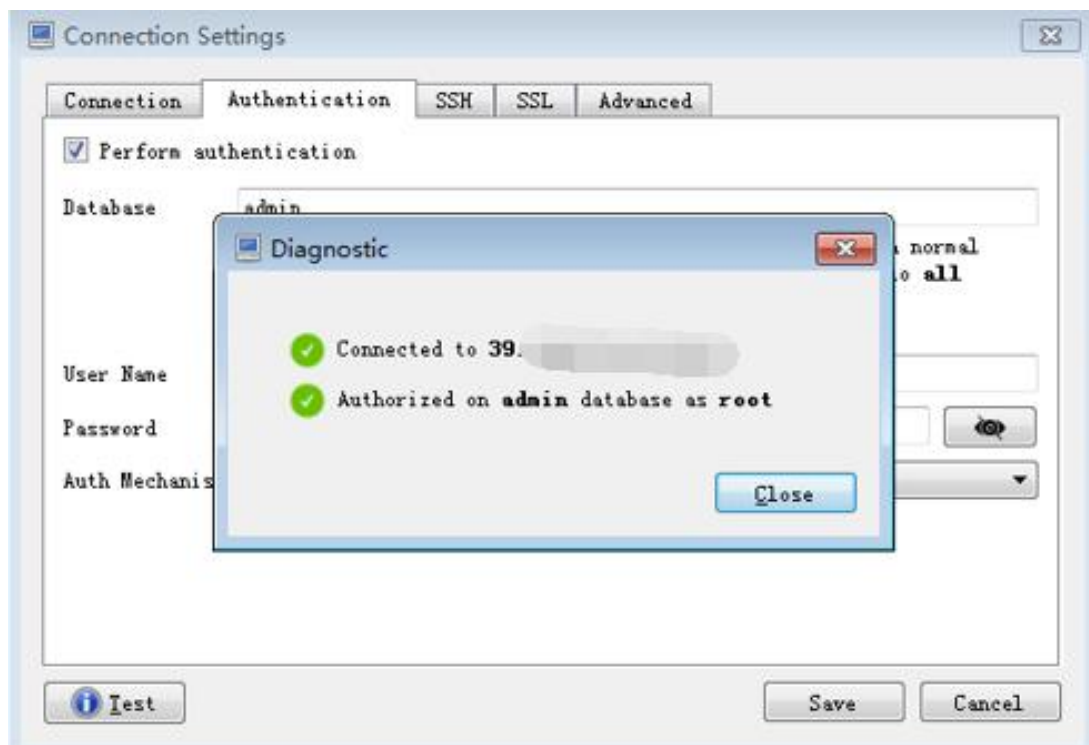




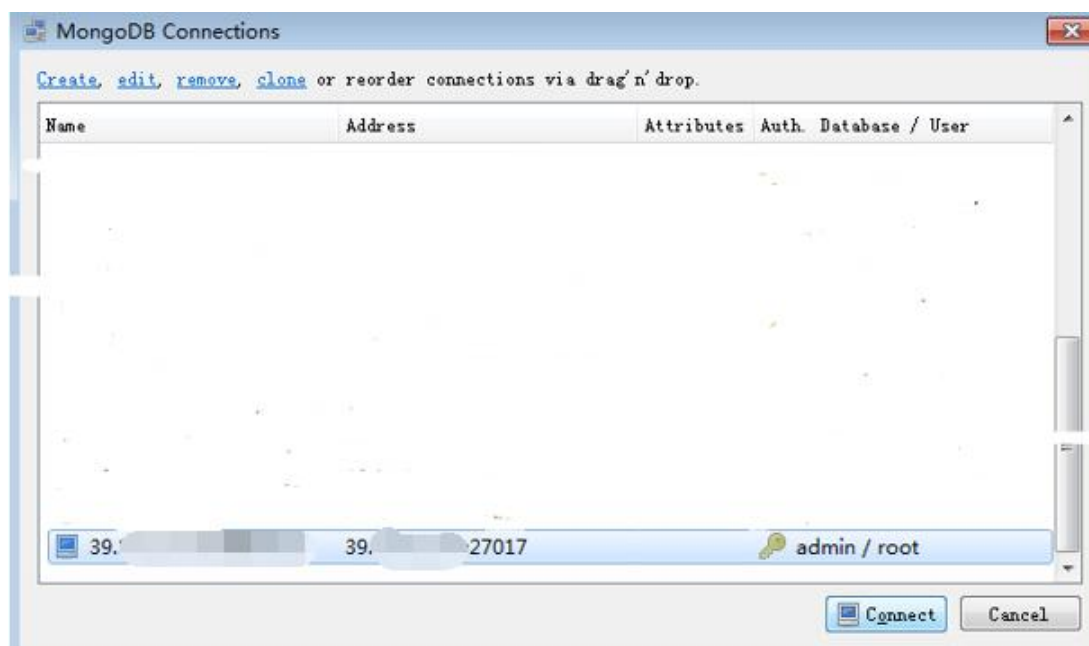
2.1.3.5 连接 mongodb

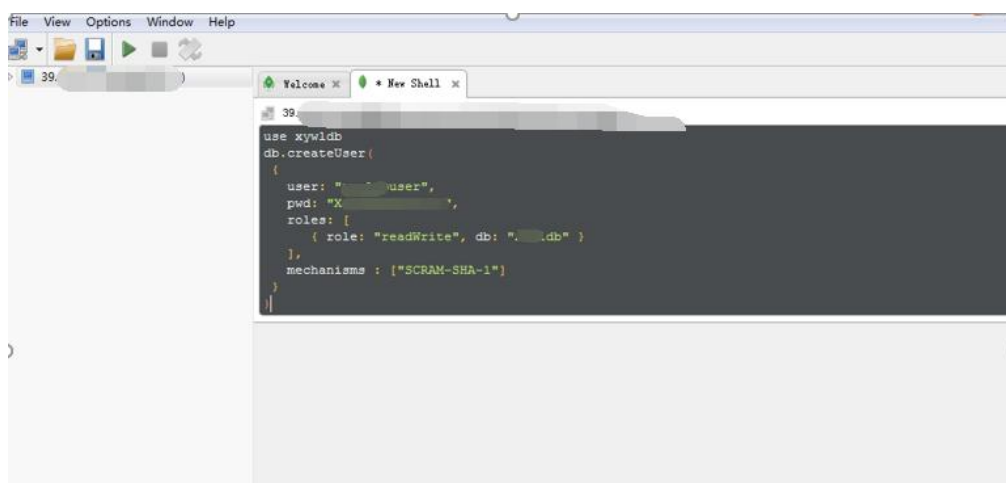
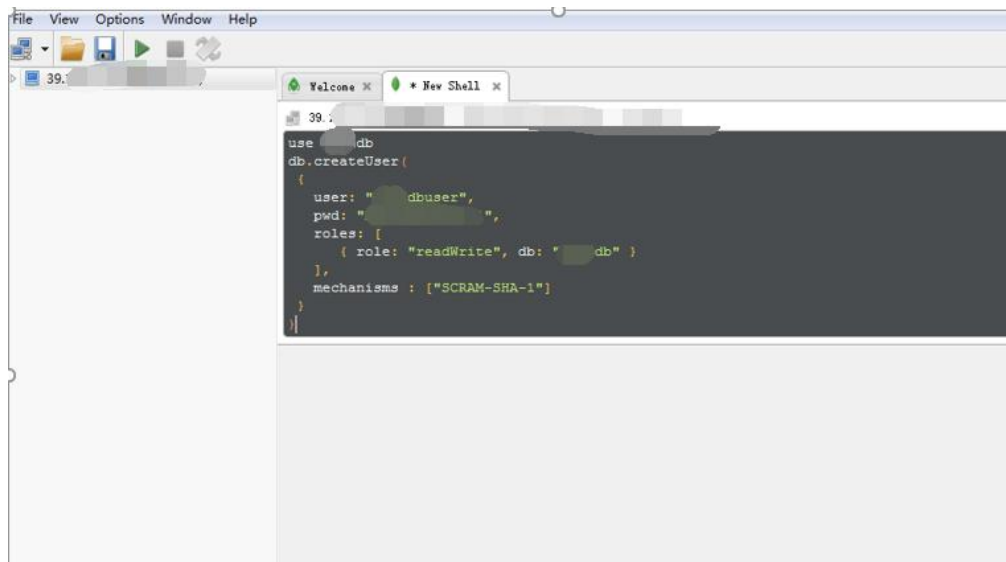
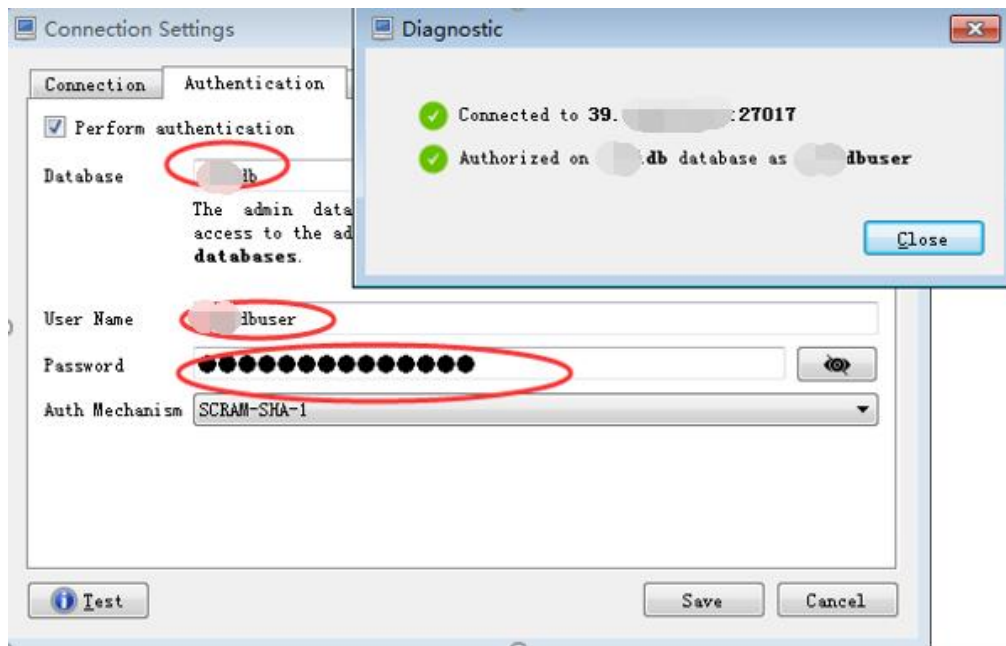
2.1.3.5.1 管理员账号登陆

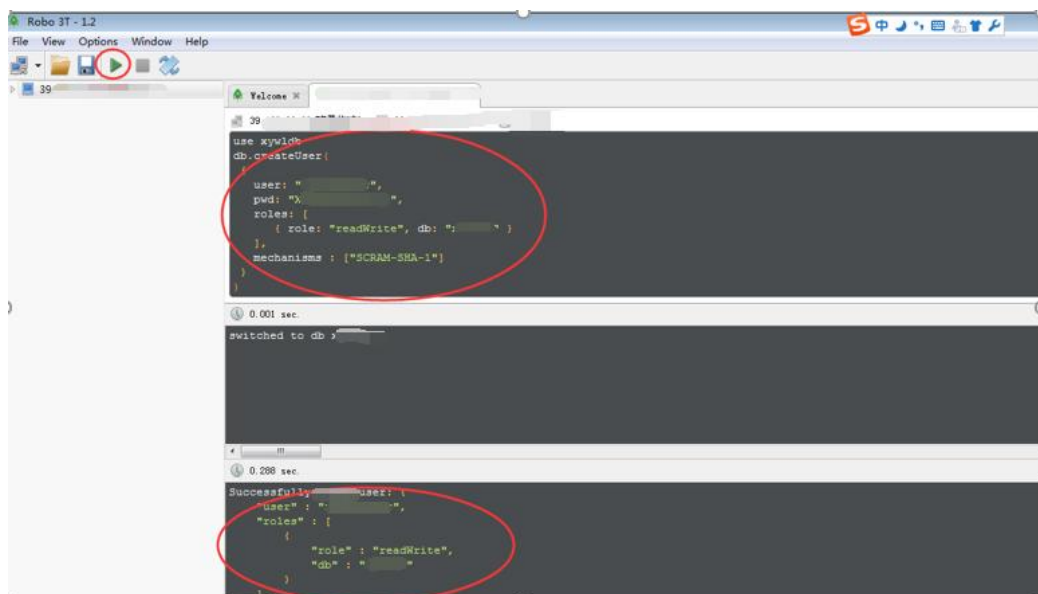




2.1.3.5.2 普通账号创建

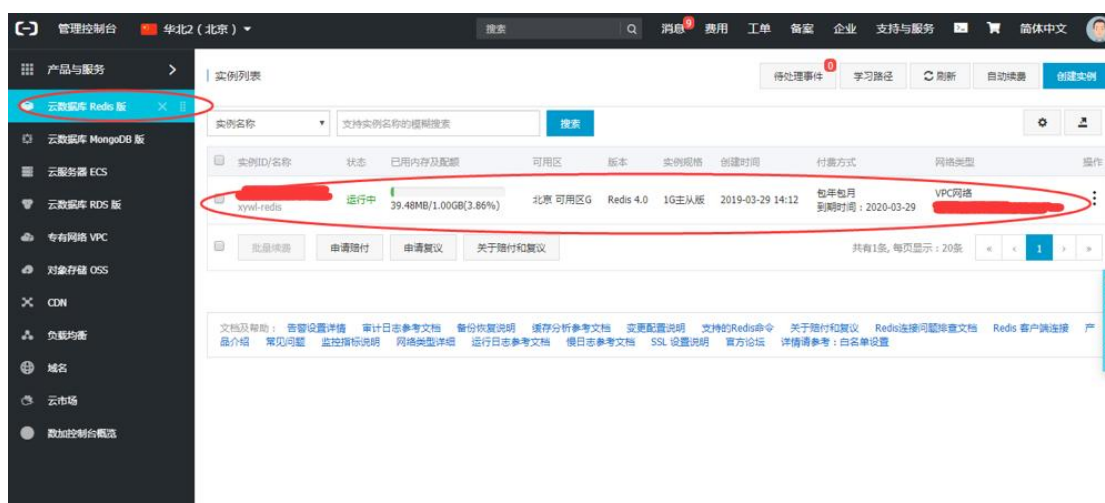






2.1.4 云数据库 Redis 设置

2.1.4.1 查看实例信息



消息⁹ 费用 工单 备案 企业 支持与服务 简体中文

待处理事件⁰ 学习路径 刷新 自动续费 创建实例

实例规格 创建时间 付费方式 网络类型 操作

1G主从版	2019-03-29 14:12	包年包月 到期时间：2020-03-29	VPC网络	...
-------	------------------	-------------------------	-------	-----

共有1条。每页显示：20条

配置说明 支持的Redis命令 关于赔付和复议 Redis连接问题排查文档
SSL 设置说明 官方论坛 详情请参考：白名单设置

管理
重启
续费
升级配置
降级配置

消息⁹ 费用 工单 备案 企业 支持与服务 简体中文

搜索 Q 消息⁹ 费用 工单 备案 企业 支持与服务 简体中文

登录数据库 迁移数据库 刷新 修改密码 清除数据 升级配置 降级配置

迁移可用区 小版本升级

实例名称：redis	状态：运行中
网络类型：VPC网络	可维护时间段：02:00-06:00 设置
交换机信息：	最大内网带宽 10 MByte 临时调整带宽
Lua状态：	最大连接数 10000

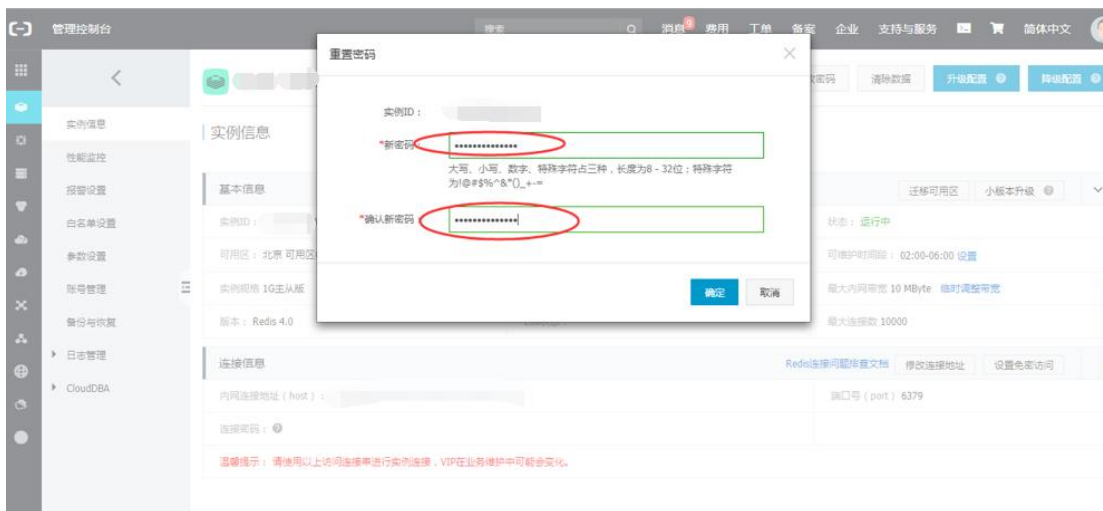
Redis连接问题排查文档 修改连接地址 设置免密访问

rds.aliyuncs.com 端口号 (port) 6379

VPC网络地址由系统自动生成



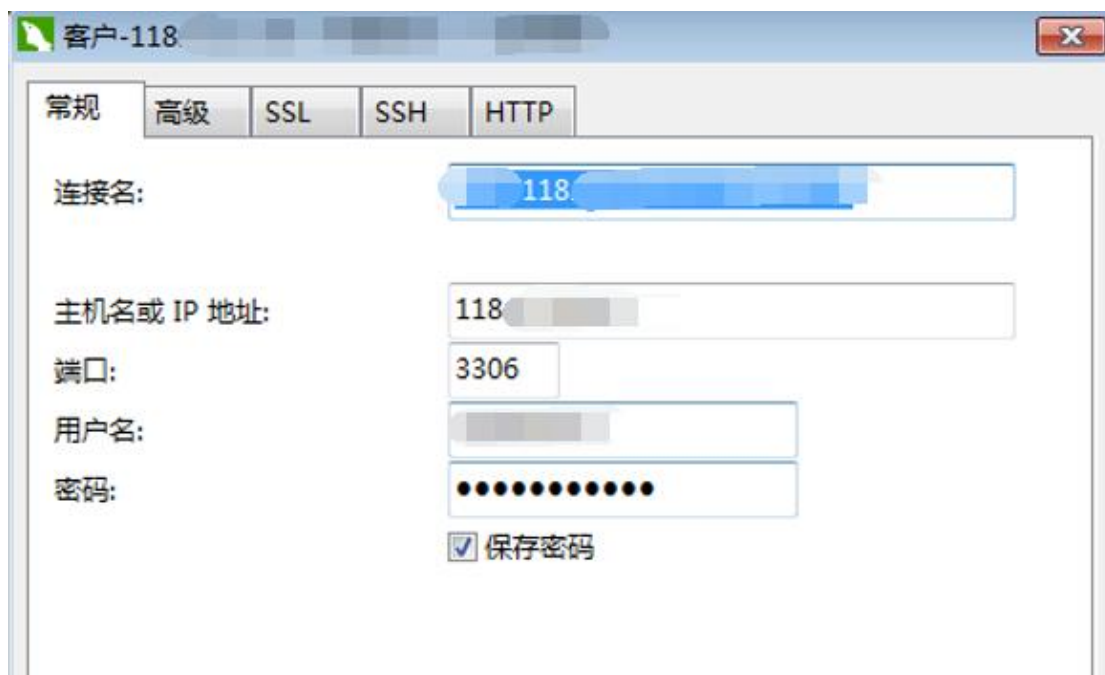
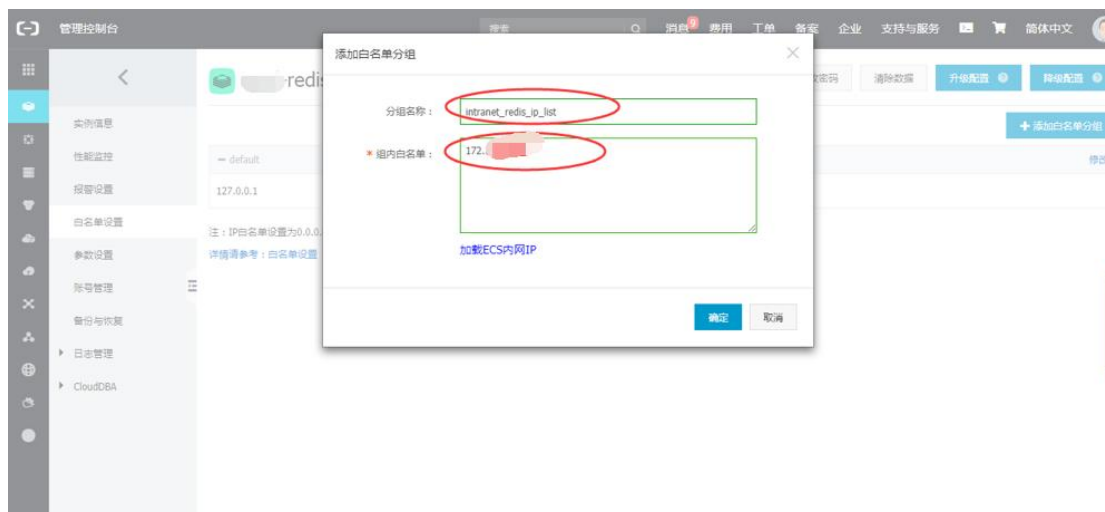
输入密码然后确认保存



2.1.4.2 白名单设置

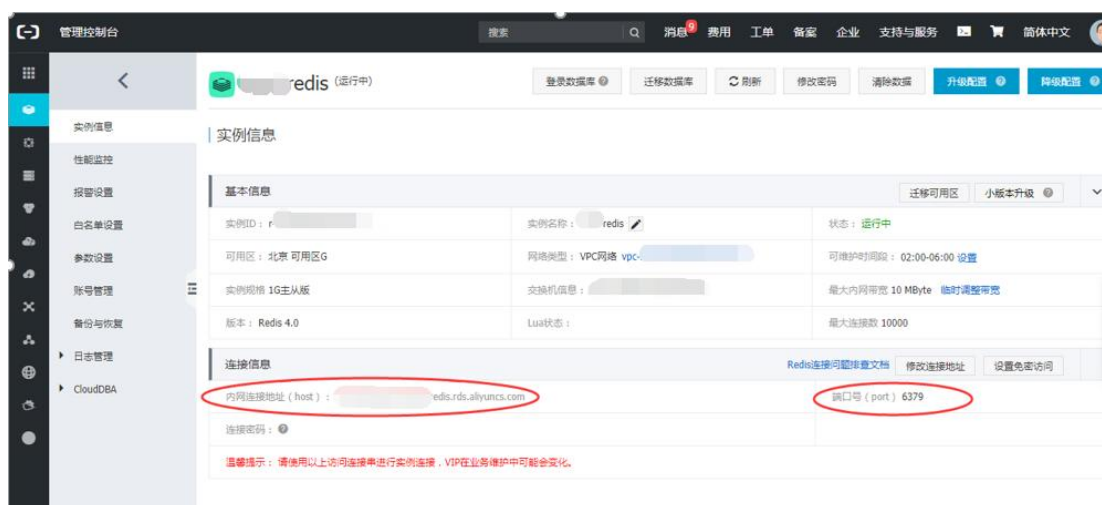
点击“白名单设置”，添加“白名单分组”





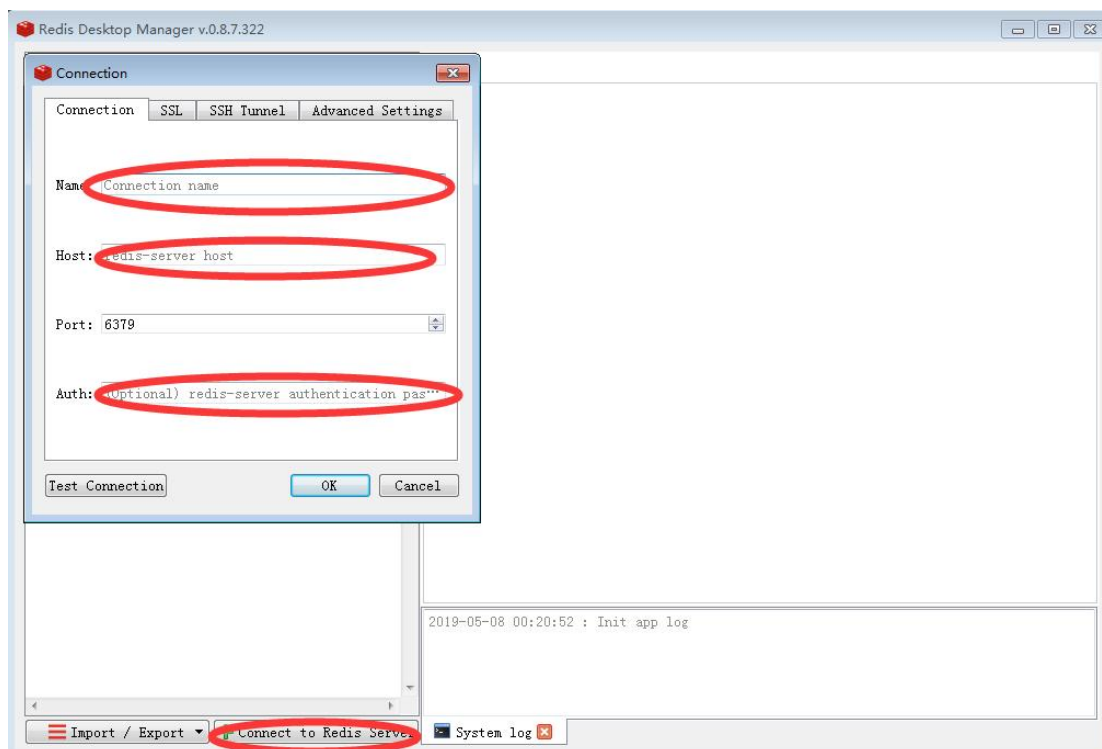
数据库初始化

2.1.4.3 查看连接信息



r-cdefghijk.redis.rds.aliyuncs.com:6379

2.1.4.4 连接 Redis



2.1.5 SSL 证书申请

2.1.5.1 SSL 证书申请

The following steps illustrate the SSL certificate purchase process on the Alibaba Cloud console:

- Overview Page:** The first screenshot shows the 'SSL 证书 (应用安全)' section in the left sidebar. The main area displays a flowchart for the 4-step process: 1. Purchase Certificate, 2. Submit Information & Apply for Certificate, 3. Domain Verification (with sub-options for DV, OV, and EV certificates), and 4. Install Certificate to Website / APP. A note indicates that DV certificates are issued within 5-10 minutes, while OV and EV certificates take 2 business days.
- Resource Package Selection:** The second screenshot shows the '立即购买' (Buy Now) modal. Under '资源包规格' (Resource Package Specifications), the '免费证书扩容包' (Free Certificate Expansion Package) is selected. The '证书个数' (Number of Certificates) is set to 20. A note explains that this package allows for a one-time issuance of up to 200 free certificates per natural person or entity per year.
- Payment Confirmation:** The third screenshot shows the '待支付订单' (Order to be Paid) modal. It displays the order details, including the '应付' (Amount Due) of ¥0.00, the '使用余额' (Use Balance) of ¥0.00, and the '其他方式支付' (Other Payment Methods) section. The '实付金额' (Actual Amount Paid) is confirmed as ¥0.00, and the '支付' (Pay) button is highlighted.

三

阿里云

账号全部资源 中国大陆

搜索文档、控制台、API、解决方案和资源

SSL证书

概览

证书资源包 new

托管服务

网站安全检测

网站代理HTTPS

信息管理

网站防篡改

有问题？找专家！

即日起至12月31日，证书资源包正式上线，全场产品7折起售，点击查看详情。

SSL证书 / 证书资源包

证书资源包

证书总数

20 个 立即购买

剩余证书数量

20 次

证书申请

已签发证书数量

0 次

证书管理

订单管理

全部状态

请输入域名

Q

证书	品牌/类型	状态	绑定域名	有效期
没有数据				

证书申请

i 托管服务即证书多年服务，通过证书自动更新和云产品自动部署实现证书多年服务。下单 **即送8次** 云产品证书部署服务，便于证书到期后自动部署到云产品。

托管服务 **?** ☐ 托管服务剩余 0 次，点击 **购买** 增加次数才能开启服务

规格

单域名

多域名

通配符



免费版 SSL

剩余消耗 20 次

本次消耗次数

- 1 +

总消耗次数

DigiCert 免费版 SSL 1次。

确定

取消

SSL证书

概览

证书资源包 new 1

托管服务

网站安全检测

网站代理+HTTPS

信息管理

网站防篡改

有问题? 找专家!

即日起至12月31日, 证书资源包正式上线, 全场产品7折起, 点击查看详情。

SSL证书 / 证书资源包

证书资源包

证书总数

20个 [立即购买](#)

剩余证书数量

19个 [证书申请](#)

已签发证书数量

1次

证书管理

订单管理

全部状态

请输入域名

Q

证书	品牌/类型	状态	绑定域名	有效期	已部署	
DigiCert 免费版 SSL	免费版 SSL	待申请	...	1年	...	证书申请 取消申请 详情

证书申请

填写申请

验证信息

* 证书绑定域名

www. com

请输入完整的单个域名, 域名格式例如: www.aliyun.com, IP证书仅Globalsign的OV单域名证书支持。

* 域名验证方式

自动DNS验证

* 联系人

com

* 所在地

中国大陆 / 上海市

* CSR生成方式

手动填写

CSR文件是您的公钥证书原始文件, 包含了您的服务器信息和您的单位信息, 需要提交给CA认证中心审核。建议您使用系统创建的CSR, 避免因内容不正确而导致的审核失败。

下一步

取消

证书申请



填写申请

验证信息

如果域名在阿里云，由域名管理员登录到域名管理控制台（dns.console.aliyun.com），按照以下提示信息添加到解析配置中，该验证记录在证书签发后才能删除，否则会因没有解析记录导致证书签发失败。

配置项目	配置项值
域名授权验证类型	DNS
记录类型	TXT ?
主机记录	_dnsauth ? 复制
记录值	20210206000001h7jzltw59mzuhstweu1el8i9802pbljlo7glemr2zwwkysrgz1 ? 复制

验证

✓ 域名验证成功，域名验证记录在证书签发后再删除，否则会因没有解析记录导致证书签发失败。

提交审核

上一步

取消

SSL证书

概览

证书资源包 new 1

托管服务

网站安全检测

网站代理HTTPS

信息管理

网站的篡改

有问题？找专家！

即日起12月31日，证书资源包正式上线，全场产品7折起，点击查看详情。

SSL证书 / 证书资源包

帮助文档

证书资源包

证书总数

20个 [立即购买](#)

剩余证书数量

19 [证书申请](#)

已签发证书数量

1次

证书管理

订单管理

全部状态

请输入域名

Q

证书	品牌/类型	状态	绑定域名	有效期限	已部署	操作
DigiCert 免费版 SSL 实例: cas-ksauto-34b4Gr	免费版 SSL	已签发	www.***.com	1年 2022年2月6日	--	部署 下载 详情

证书下载



请根据您的服务器类型选择证书下载：

服务器类型	操作	
Tomcat	帮助	下载
Apache	帮助	下载
Nginx	帮助	下载
IIS	帮助	下载
JKS	帮助	下载
其他	下载	
根证书下载	下载	

一键式HTTPS服务

不用安装证书，不用纠结各种安全套件的选择，不用担心私钥泄露，一键式HTTPS服务帮助 [立即使用](#) 您解决网站HTTPS问题。



下载的文件如下：5171649_www.abc.com_nginx.zip，解压后文件如下：

```

5171649_www.abc.com.key
5171649_www.abc.com.pem
    
```

2.1.6 系统内核调优(参考)

```

$ vim /etc/sysctl.conf
vm.swappiness = 0
kernel.sysrq = 1
net.ipv4.neigh.default.gc_stale_time = 120
net.ipv4.conf.all.rp_filter = 0
net.ipv4.conf.default.rp_filter = 0
    
```

```
net.ipv4.conf.default.arp_announce = 2
net.ipv4.conf.lo.arp_announce = 2
net.ipv4.conf.all.arp_announce = 2
net.ipv4.tcp_max_tw_buckets = 5000
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 1024
net.ipv4.tcp_synack_retries = 2
net.ipv4.tcp_slow_start_after_idle = 0
net.ipv4.ip_local_port_range = 10240 65000
fs.file-max = 655350000
fs.nr_open = 655350000
net.ipv4.ip_forward = 1
```

必须先执行 `sysctl -p` 才可以往下修改 `limitd.conf`；顺序反了系统会无法登陆等异常

```
$ sysctl -p
$ vim /etc/security/limitd.conf
root soft nofile 65535000
root hard nofile 65535000
* soft nofile 65535000
* hard nofile 65535000
$vim /etc/security/limits.d/20-nproc.conf
*          soft      nproc      4096000
root       soft      nproc      unlimited
```

2.1.7 应用目录创建

```
$ mkdir -p /data/app/www/v2.0/{download,xiaonih5,appimg}
# app 下载目录/data/app/www/v2.0/download；对应nginx里location /download/里root
目录；存放打好包的安卓app；提供下载
# h5 目录/data/app/www/v2.0/niuxweb；将web源码构建后dist/目录下的文件传到niuxweb
目录下提供访问
#appimg 小程序依赖目录
# 目录/data/bin/restart-server-niux.sh 手工启动应用服务的脚本；如不需要脚本启动可以
手动通过命令启动；脚本启动见下方“jar 启动脚本配置说明”；启动应用的帐户需要对此目录有读写权限
$ mkdir /data/logs
# nginx 配置ssl证书的默认目录；将证书传到此目录下并修改nginx的ssl crt和key的文件名；
目录可更改
$ mkdir /data/openssl/cert -p
#存放图片合同类文件，并提供nginx展示
#ls /data/app/static
#/data/website_test，目录存放构建完的jar包；与下方“jar 启动脚本配置说明”相对应的代
码目录，启动jar的脚本restart-server-niux.sh niux-gateway 会从/data/website_test 目录
下读niux-gateway.jar来启动；启动应用的帐户需要对此目录有读写权限
#mkdir /data/website_test
```

2.1.8 存储图片目录配置

主要用于存储图片，配置如下：

```
$ mkdir /data/app/static
```

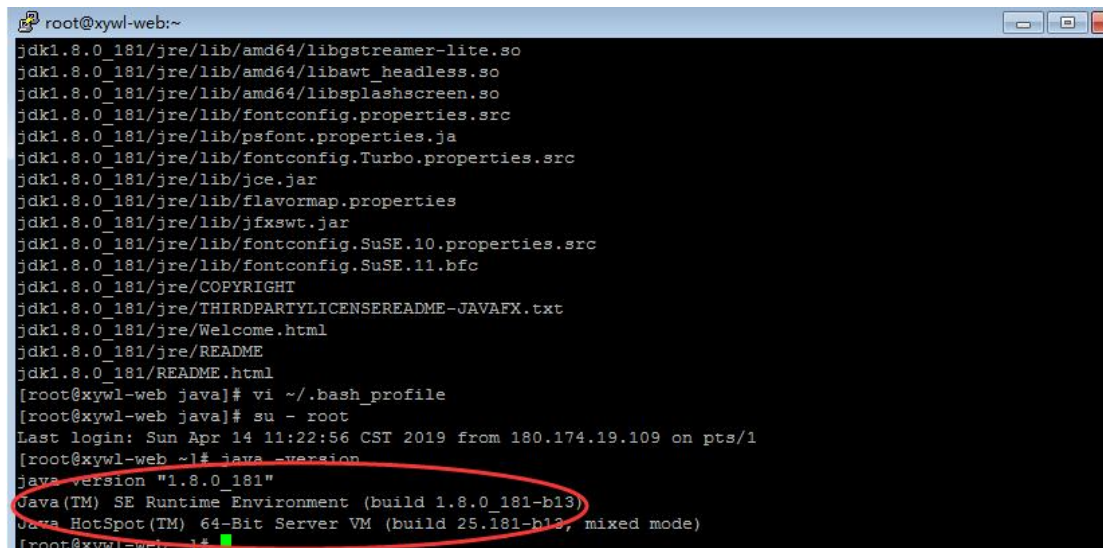
2.1.9 Jdk 安装

1) 将下载好的jdk安装包jdk-8u181-linux-x64.tar.gz存放于/data/app目录下。

```
$ mkdir /usr/java
$ cd /data/app
$ cp jdk-8u181-linux-x64.tar.gz /usr/java
$ cd /usr/java
$ tar -xvzf jdk-8u181-linux-x64.tar.gz
$ vi ~/.bash_profile
    export JAVA_HOME=/usr/java/jdk1.8.0_181
    export PATH=$JAVA_HOME/bin:$PATH
$ source ~/.bash_profile
```

2) 验证

```
$ java -version
```



```
root@xywl-web:~
jdk1.8.0_181/jre/lib/amd64/libgstreamer-lite.so
jdk1.8.0_181/jre/lib/amd64/libawt_headless.so
jdk1.8.0_181/jre/lib/amd64/libsplashscreen.so
jdk1.8.0_181/jre/lib/fontconfig.properties.src
jdk1.8.0_181/jre/lib/psfont.properties.ja
jdk1.8.0_181/jre/lib/fontconfig.Turbo.properties.src
jdk1.8.0_181/jre/lib/jce.jar
jdk1.8.0_181/jre/lib/flashmap.properties
jdk1.8.0_181/jre/lib/jfxswt.jar
jdk1.8.0_181/jre/lib/fontconfig.SuSE.10.properties.src
jdk1.8.0_181/jre/lib/fontconfig.SuSE.11.bfc
jdk1.8.0_181/jre/COPYRIGHT
jdk1.8.0_181/jre/THIRDPARTYLICENSEREADME-JAVAFX.txt
jdk1.8.0_181/jre/Welcome.html
jdk1.8.0_181/jre/README
jdk1.8.0_181/README.html
[root@xywl-web java]# vi ~/.bash_profile
[root@xywl-web java]# su - root
Last login: Sun Apr 14 11:22:56 CST 2019 from 180.174.19.109 on pts/1
[root@xywl-web ~]# java -version
java version "1.8.0_181"
Java(TM) SE Runtime Environment (build 1.8.0_181-b13)
Java HotSpot(TM) 64-Bit Server VM (build 25.181-b13, mixed mode)
[root@xywl-web ~]#
```

2.1.10Nginx 安装

1) 安装 nginx

安装 Nginx 依赖

```
$ yum -y install mailx nss
```

安装 Nginx, 用作代理服务器整合 tms 与各个组件

```
$ wget https://niuinfotms.oss-cn-hangzhou.aliyuncs.com/soft/nginx-1.25.3-0.x86_64.rpm
```

```
$ rpm -ivh nginx-1.25.3-0.x86_64.rpm
```

```
$ systemctl enable nginx
```

2) 配置 nginx

/etc/nginx/conf.d 目录下新增 tms.conf 如下:

```
$ vi /etc/nginx/conf.d/tms.conf
```

#配置访问服务器, 修改实际服务端口 6114=niux-activiti.jar 6101=niux-gateway.jar
6123=niux-xxljob.jar

```
upstream prodactiviti {  
    server 127.0.0.1:6114;  
}  
upstream prodgateway {  
    server 127.0.0.1:6101;  
}  
upstream prodxxljob {  
    server 127.0.0.1:6123;  
}
```

#修改实际的域名

```
server {  
    server_name www.abc.cn;  
    #如使用https 访问, 将下方 ssl_certificate 和 ssl_certificate_key 证书文件换成  
    实际 文件名; 并且上传到配置的目录下  
    listen 443 ssl;  
    #如使用http 方式访问: 将上方 443 ssl 改成 80  
    #如使用http 方式访问: 将下方 ssl 注释;  
    index index.html index.htm index.php;  
    # 配置 SSL 证书路径 (参考 SSL 证书的获取方式), 上传证书到  
    /data/openssl/cert/(可修改) 并配置证书文件名  
    ssl_certificate /data/openssl/cert/5171649_www.abc.com.pem;  
    ssl_certificate_key /data/openssl/cert/5171649_www.abc.com.key;  
    ssl_session_timeout 5m;
```

```
ssl_ciphers ECDHE-RSA-AES128-GCM-
SHA256:ECDHE:ECDH:AES:HIGH:!NULL:!aNULL:!MD5:!ADH:!RC4:!DES:!des-cbc3-sha:!ECDHE-
RSA-DES-CBC3-SHA;;

ssl_protocols TLSv1.2;
ssl_prefer_server_ciphers on;
root /data/app/www/v2.0/niuxweb;
client_max_body_size 300m;

# 支持跨越访问
add_header Access-Control-Allow-Origin *;

# 配置 web 访问
location / {
    proxy_set_header Host $http_host:$server_port;
    # 用于配合 browserHistory 使用
    try_files $uri $uri/ /index.html;
}

# 配置接口访问, 6114=niux-activiti.jar 服务
location /jeecgboot {
    # 配置反向代理
    proxy_pass http://prodactiviti;
    proxy_redirect off;
    #真实 IP
    proxy_set_header Host $http_host:$server_port;
    proxy_set_header X-Real-IP $remote_addr;
    set $my_proxy_add_x_forwarded_for $proxy_add_x_forwarded_for;
    if ($proxy_add_x_forwarded_for ~* "127.0.0.1"){
        set $my_proxy_add_x_forwarded_for $remote_addr;
    }
    proxy_set_header X-Forwarded-For $my_proxy_add_x_forwarded_for;
}

# 配置接口访问, 6101=niux-gateway.jar 服务
location /sys/thirdLogin/wechat_open/callback {
    proxy_pass http://prodgateway/sys/thirdLogin/wechat\_open/callback;
    proxy_redirect off;
    #真实 IP
    proxy_set_header Host $http_host:$server_port;
    proxy_set_header X-Real-IP $remote_addr;
    set $my_proxy_add_x_forwarded_for $proxy_add_x_forwarded_for;
    if ($proxy_add_x_forwarded_for ~* "127.0.0.1"){
        set $my_proxy_add_x_forwarded_for $remote_addr;
    }
    proxy_set_header X-Forwarded-For $my_proxy_add_x_forwarded_for;
```

```

}
# 配置接口访问, 6101=niux-gateway.jar 服务
location /sys/thirdLogin/dingtalk/callback {
    proxy_pass http://prodgateway/sys/thirdLogin/dingtalk/callback;
    proxy_redirect off;
    #真实 IP
    proxy_set_header Host $http_host:$server_port;
    proxy_set_header X-Real-IP $remote_addr;
    set $my_proxy_add_x_forwarded_for $proxy_add_x_forwarded_for;
    if ($proxy_add_x_forwarded_for ~* "127.0.0.1"){
        set $my_proxy_add_x_forwarded_for $remote_addr;
    }
    proxy_set_header X-Forwarded-For $my_proxy_add_x_forwarded_for;

}

# 配置接口访问
location /sys/thirdLogin/wechat_enterprise/callback {
    proxy_pass
http://prodgateway/sys/thirdLogin/wechat\_enterprise/callback;
    proxy_redirect off;
    #真实 IP
    proxy_set_header Host $http_host:$server_port;
    proxy_set_header X-Real-IP $remote_addr;
    set $my_proxy_add_x_forwarded_for $proxy_add_x_forwarded_for;
    if ($proxy_add_x_forwarded_for ~* "127.0.0.1"){
        set $my_proxy_add_x_forwarded_for $remote_addr;
    }
    proxy_set_header X-Forwarded-For $my_proxy_add_x_forwarded_for;

}

location /xxl-job-admin/ {
    proxy_pass http://prodxxljob/xxl-job-admin/;
    proxy_set_header Host $http_host:$server_port;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-Scheme $scheme;

}

# 配置手机端H5 访问路径
location /niux/ {
    root /data/app/www/v2.0/niuxapp ;

```

```

    }

    location /download/ {
        root /data/app/www/v2.0;
    }
# 配置小程序文本验证
    location ~*.(txt)$ {
        root /data/app/www/v2.0;
    }
# 配置apk 下载路径
    location ~* .(mp4|mov)$ {
        root /data/app/www/v2.0/vedio;
    }
# 配置小程序图片路径
    location /appimg/ {
        root /data/app/www/v2.0;
    }
}
# 以上 tms.conf 配置结束

```

/etc/nginx/conf.d 目录下新增api.tms.conf 如下:

```
$ vi /etc/nginx/conf.d/api.tms.conf
```

#配置访问服务器，实际服务端口 6114=niux-activiti.jar 6101=niux-gateway.jar 已经配置在tms.conf，这里不可再重复配置

#修改实际的域名

```

server {
    server_name api.abc.cn;
    #如使用https 访问,将下方ssl_certificate 和ssl_certificate_key 证书文件换成
    实际 文件名; 并且上传到配置的目录下
    listen 443 ssl;
    #如使用http 方式访问: 将上方443 ssl 改成80
    #如使用http 方式访问: 将下方ssl 注释;
    index index.html index.htm index.php;
    # 配置SSL 证书路径(参考SSL 证书的获取方式), 上传证书到
    /data/openssl/cert/(可修改) 并配置证书文件名
    ssl_certificate /data/openssl/cert/5171649_api.abc.com.pem;
    ssl_certificate_key /data/openssl/cert/5171649_api.abc.com.key;
    ssl_session_timeout 5m;
    ssl_ciphers ECDHE-RSA-AES128-GCM-
    SHA256:ECDHE:ECDH:AES:HIGH:!NULL:!aNULL:!MD5:!ADH:!RC4:!DES:!des-cbc3-sha:!ECDHE-
    RSA-DES-CBC3-SHA;;

```

```
ssl_protocols TLSv1.2;
ssl_prefer_server_ciphers on;
client_max_body_size 50m;
client_body_timeout 50s;
client_body_buffer_size 128k;

# 支持跨越访问
add_header Access-Control-Allow-Origin *;
# 配置 niux-gateway.jar 访问
location / {
    root    html;
    index  index.html index.htm;
    proxy_pass    http://prodgateway;
    proxy_set_header    X-Forwarded-Scheme    $scheme;
    proxy_redirect      off;
    proxy_set_header    Host    $http_host:$server_port;
    proxy_set_header    X-Real-IP        $remote_addr;
    proxy_set_header    X-Forwarded-For  $proxy_add_x_forwarded_for;

}

# 配置接口访问, niux-activiti.jar 服务
location /websocket/ {
    # 配置反向代理
    proxy_pass          http://prodactiviti/websocket/;
    proxy_set_header    X-Forwarded-Scheme    $scheme;
    proxy_redirect      off;
    proxy_set_header    Host    $http_host:$server_port;
    proxy_set_header    X-Real-IP        $remote_addr;
    proxy_set_header    X-Forwarded-For  $proxy_add_x_forwarded_for;
}

# 配置接口访问, niux-activiti.jar 服务
location /druid {
    proxy_pass http://prodactiviti/druid;
    break;
}

# 配置小程序文本验证
location ~*(.txt)$ {
    root /data/app/www/v2.0;
}

}

# api.tms.conf 以上配置结束
```

3) 启动 nginx

```
$ nginx -t
$ systemctl restart nginx
```

2.1.11 Nacos 安装

1) 安装 nacos

```
$ cd /usr/local/src/
$ wget https://nacos-server-2.2.3.tar.gz
```

2) 配置 nacos

```
$ tar -zxvf nacos-server-2.2.3.tar.gz
$ cp nacos-server/data/app/nacos/conf
$ cd /data/app/nacos/conf
$ vim application.properties
```

```
server.servlet.contextPath=/nacos
server.error.include-message=ALWAYS
server.port=8848
db.pool.config.connectionTimeout=30000
db.pool.config.validationTimeout=10000
db.pool.config.maximumPoolSize=20
db.pool.config.minimumIdle=2
management.metrics.export.elastic.enabled=false
management.metrics.export.influx.enabled=false
server.tomcat.accesslog.enabled=true
server.tomcat.accesslog.pattern=%h %l %u %t "%r" %s %b %D %{User-Agent}i %{Request-Source}i
server.tomcat.basedir=file:.
nacos.security.ignore.urls=,/error,/**/*.css,/**/*.js,/**/*.html,/**/*.map,/**/*.svg,/**/*.png,/**/*.ico,/console-
ui/public/**/*.v1/auth/**/*.v1/console/health/**/*.actuator/**/*.v1/console/server/**/*.v1
nacos.core.auth.system.type=nacos
nacos.core.auth.enabled=true
nacos.core.auth.caching.enabled=true
nacos.core.auth.enable.userAgentAuthWhite=true
nacos.core.auth.server.identity.key=nacos
nacos.core.auth.server.identity.value=nacos
nacos.core.auth.plugin.nacos.token.cache.enable=false
nacos.core.auth.plugin.nacos.token.expire.seconds=18000
nacos.core.auth.plugin.nacos.token.secret.key=KFhDaBRTumJXLwHxImmvHYAYnxsQdpVjAN
PcVAdTnjkyuAKJrCmitwKpwNCfLfHDtcawhGRzrGjWxpecsSYQaZmLdBuDWhhqAVPCANVcF
OsqxWPvfNINFIPwrVjQPZhdc
nacos.istio.mcp.server.enabled=false
```

3) 启动 nacos

```
$ cd /data/app/nacos/bin
$ ./startup.sh -m standalone
$ echo 'bash /data/app/nacos/bin/startup.sh -m standalone > /dev/null 2>&1' >>
/etc/rc.d/rc.local
```

4) 访问 nacos

浏览器打开 <http://ip:8848/>

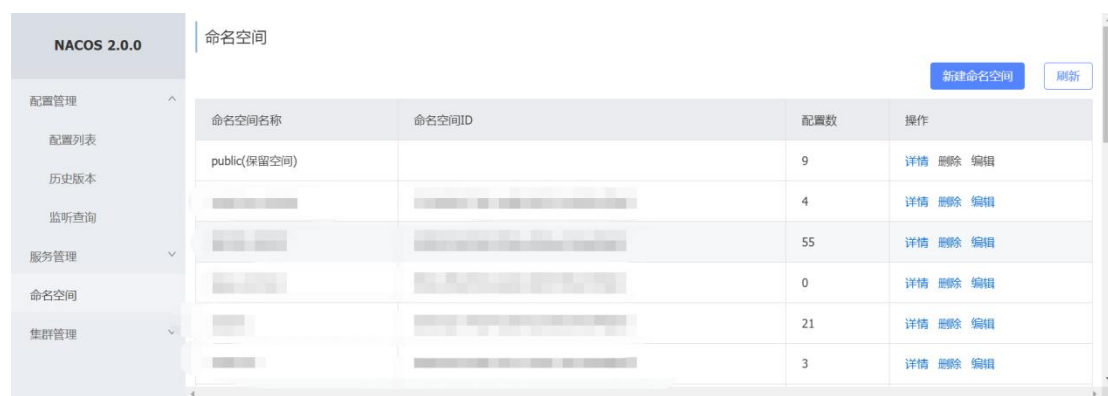
默认帐号 nacos

默认密码 nacos

及时修改默认密码

5) 创建命名空间

选择左边命名空间



命名空间名称	命名空间ID	配置数	操作
public(保留空间)		9	详情 删除 编辑
		4	详情 删除 编辑
		55	详情 删除 编辑
		0	详情 删除 编辑
		21	详情 删除 编辑
		3	详情 删除 编辑



NACOS 2.2.3

配置管理



配置列表

历史版本

监听查询

服务管理



权限控制



命名空间

集群管理



选择右上角 新建命名空间

新建命名空间

刷新

填写内容： 命名空间 id: test 命名空间名: test 描述: test

新建命名空间



命名空间ID(不填则

test

自动生成):

* 命名空间名:

test

* 描述:

test

确定

取消

6) 上传配置文件 nacos_test_demo_niux.zip 到命名空间



nacos_test_demo_niux.zip

备注: zip 文件从商务处获取

选择左边配置管理---->配置列表---->选择上面新建出来的命名空间 test

选择导入配置----->上传文件



7) 修改上传的配置文件

- 每个配置文件中的数据库连接地址，端口，帐号，密码修改为实际的信息
- 域名修改为实际的域名

例子如下

修改 niux-gateway-prod.yaml 红色部分

```
niux:
  route:
    config:
      #type:database nacos yaml
      data-type: database
      group: DEFAULT_GROUP
      data-id: niux-gateway-router
spring:
  redis:
    # database: 0
    # host: niux-boot-redis
    # port: 6379
    # password: redispass
    password: redispass
    database: 0
  lettuce:
    pool:
```

```
    max-active: 10
    max-wait: 300
    max-idle: 16
    min-idle: 8
  cluster:
    max-redirects: 3
    nodes:
      - 172.18.187.11:13379
      - 172.18.187.12:13379
      - 172.18.187.13:13379
      - 172.18.187.11:14379
      - 172.18.187.12:14379
      - 172.18.187.13:14379
```

修改 niux-prod.yaml 红色部分

```
spring:
  servlet:
    multipart:
      max-request-size: 500MB
      max-file-size: 500MB
  datasource:
    druid:
      stat-view-servlet:
        enabled: true
        loginUsername: admin
        loginPassword: druidadminpass
        allow:
      web-stat-filter:
        enabled: true
  dynamic:
    druid:
      initial-size: 5
      min-idle: 5
      maxActive: 100
      maxWait: 60000
      timeBetweenEvictionRunsMillis: 60000
      minEvictableIdleTimeMillis: 300000
      validationQuery: SELECT 1 FROM DUAL
      testWhileIdle: true
      testOnBorrow: false
      testOnReturn: false
      poolPreparedStatements: true
      maxPoolPreparedStatementPerConnectionSize: 100
```

```
    filters: stat,wall,slf4j
    connectionProperties:
druid.stat.mergeSql=true;druid.stat.slowSqlMillis=5000
    remove-abandoned: false
    remove-abandoned-timeout-millis: 1800
    log-abandoned: true
datasource:
  master:
    url:
jdbc:mysql://172.18.188.4:3306/mysql?characterEncoding=UTF-
8&useUnicode=true&useSSL=false&tinyInt1isBit=false&allowPublicKeyRetrie
val=true&serverTimezone=Asia/Shanghai&nullCatalogMeansCurrent=true&zero
DateTimeBehavior=CONVERT_TO_NULL
    username: mysqluser
    password: mysqlpass
    driver-class-name: com.mysql.cj.jdbc.Driver
redis:
  # database: 0
  # host: niuxboot-redis
  # password: www.tewe23stteste
  # port: 6379
password: redispass
database: 0
lettuce:
  pool:
    max-active: 10
    max-wait: 300
    max-idle: 16
    min-idle: 8
cluster:
  max-redirects: 3
  nodes:
    - 172.18.187.11:13379
    - 172.18.187.12:13379
    - 172.18.187.13:13379
    - 172.18.187.11:14379
    - 172.18.187.12:14379
    - 172.18.187.13:14379
data:
  mongodb:
    host: 172.18.188.4
    port: 27017
    database: mongodatabase
    username: mongouser
```

```
    password: mongopass
rabbitmq:
  host: 172.18.187.88
  username: guest
  password: guest
  port: 5672
  publisher-confirms: true
  publisher-returns: true
  virtual-host: /
  listener:
    simple:
      acknowledge-mode: manual
      concurrency: 1
      max-concurrency: 1
      retry:
        enabled: false
  main:
    allow-bean-definition-overriding: true
    allow-circular-references: true
minidao:
  base-package: org.jeecg.modules.jmreport.*,org.jeecg.modules.drag.*
jeecg:
  signatureSecret: dd05f1c54d63749eda95f9fa6d49v442a
  signUrls:
/sys/dict/getDictItems/*,/sys/dict/loadDict/*,/sys/dict/loadDictOrderBy
Value/*,/sys/dict/loadDictItem/*,/sys/dict/loadTreeData,/sys/api/queryT
ableDictItemsByCode,/sys/api/queryFilterTableDictInfo,/sys/api/queryTab
leDictByKeys,/sys/api/translateDictFromTable,/sys/api/translateDictFrom
TableByKeys
  # uploadType: alioss
  uploadType: local
  domainUrl:
    pc: http://localhost:3100
    app: http://localhost:8051
  path:
    upload: /data/app/static/upload
    webapp: /opt/webapp
  shiro:
    excludeUrls:
/test/jeecgDemo/demo3,/test/jeecgDemo/redisDemo/**,/category/**,/visual
/**,/map/**,/jmreport/bigscreen2/**,/openAPI/**,/sys/loginByOpenApi
  oss:
    endpoint: oss-cn-beijieng.aliyuncs.com
    accessKey: xxxxx
```

```

secretKey: xxxxx
bucketName: xxxxx
staticDomain: oss-cn-beijing.aliyuncs.com
elasticsearch:
  cluster-name: jeecg-ES
  cluster-nodes: niux-boot-es:9200
  check-enabled: false
file-view-domain: 127.0.0.1:8012
minio:
  minio_url: http://minio.niux.com
  minio_name: ??
  minio_pass: ??
  bucketName: otatest
jmreport:
  mode: dev
  is_verify_token: false
  verify_methods: remove,delete,save,add,update
  saas: true
  openTenant: true
  #多租户模式，默认值为空(created:按照创建人隔离、tenant:按照租户隔离)
(v1.6.2+ 新增)
  saasMode: tenant
  # 平台上线安全配置(v1.6.2+ 新增)
  firewall:
    # 数据源安全（开启后，不允许使用平台数据源、SQL 解析加签并不允许查询数据库）
    dataSourceSafe: false
    # 低代码开发模式（dev:开发模式，prod:发布模式-关闭在线报表设计功能，分配角色 admin、lowdeveloper 可放开限制）
    lowCodeMode: dev
wps:
  domain: https://wwo.wps.cn/office/
  appid: ??
  appsecret: ??
xxljob:
  enabled: true
  adminAddresses: http://127.0.0.1:6123/xxl-job-admin
  appname: ${spring.application.name}
  accessToken: ''
  logPath: logs/niux/job/jobhandler/
  logRetentionDays: 30
redisson:
  address: 172.18.188.4:6379
  password: redispass

```

```
    type: STANDALONE
    enabled: true
logging:
  level:
    com.niuinfo.modules.system.mapper : info
cas:
  prefixUrl: http://localhost:8888/cas
knife4j:
  production: false
  basic:
    enable: false
    username: jeecg
    password: jeecg1314
justauth:
  enabled: true
  type:
    GITHUB:
      client-id: ??
      client-secret: ??
      redirect-uri: http://sso.test.com:8080/niux-
boot/thirdLogin/github/callback
    WECHAT_ENTERPRISE:
      client-id: WECHAT-client-id
      client-secret: WECHAT-client-secret
      redirect-uri:
https://www.abc.com/sys/thirdLogin/wechat_enterprise/callback
      agent-id: 1000028
    DINGTALK:
      client-id: ding-client-id
      client-secret: ding-client-secret
      redirect-uri:
https://www.abc.com/sys/thirdLogin/dingtalk/callback
    WECHAT_OPEN:
      client-id: WECHAT-client-id
      client-secret: WECHAT-client-secret
      redirect-uri:
https://www.abc.com/sys/thirdLogin/wechat_open/callback
  cache:
    type: default
    prefix: 'demo::'
    timeout: 1h
third-app:
  enabled: true
  type:
```

```
WECHAT_ENTERPRISE:
  enabled: true
  client-id: WECHAT-client-id
  client-secret: WECHAT-client-secret
  agent-id: 1000028
DINGTALK:
  enabled: false
  client-id: ??
  client-secret: ??
  agent-id: ??
# apmp:
#   key: testswer
apmp:
  serviceKey: apmpserviceKey
  webKey: apmpwebkey
  webSecurityCode: apmpwebsecuritycode
mybatis:
  mapper-locations: classpath:mapper/*.xml
  map-underscore-to-camel-case: true
  # type-aliases-package: com.xxx.xxx.entity
  type-handlers-package: org.jeecg.config.mybatis.MySqlJsonHandler

fonts:
  path: /data/bin/config/fonts/
ortools:
  lib:
    path: /data/bin/ortools/

niux:
  openAPI:
    timeToken: 7200
    excludeUrls: /openAPI/auth/getAccessToken
    url: http://127.0.0.1:6119
    appId: 1234abcdwwa
    appSecret: 1234abcdwwa
  export:
    limit: 10000
seata:
  enabled: true
  enable-auto-data-source-proxy: true
  data-source-proxy-mode: AT
  tx-service-group: default_tx_group
  service:
    grouplist:
```

```
    default: 172.18.188.3:8091
    vgroup-mapping:
        default_tx_group: default
eqb:
    host: https://openapi.esign.cn
canal:
    server-ip: 172.18.187.88
    port: 11111
    user: canaluser
    password: canalpass
    destination: niux
    batch-size: 5120
server:
    tomcat:
        threads:
            #最大工作线程数量
            max: 800
            #最小工作线程数量
            min-spare: 50
            #等待队列长度
        accept-count: 100
```

2.1.12 Rabbitmq 安装

1) 安装 Docker

```
$ vim /etc/yum.repos.d/docker-ce.repo
[docker-ce-stable]
name=Docker CE Stable - $basearch
baseurl=https://download.docker.com/linux/centos/$releasever/$basearch/stable
enabled=1
gpgcheck=1
gpgkey=https://download.docker.com/linux/centos/gpg
$ yum -y install docker-ce
```

2) 部署 Rabbitmq

```
$ docker run -dit --name rabbitmq3.7.7 -e RABBITMQ_DEFAULT_USER=guest -e
RABBITMQ_DEFAULT_PASS=guest -p 15672:15672 -p 5672:5672 rabbitmq:3.7.7-management
```

2.1.13 Canal 安装

1) 安装 Canal

```
$ mkdir -p /data/app/canal
```

```
$ cd /data/app/canal
$ wget https://github.com/alibaba/canal/releases/download/canal-1.1.6/canal.deployer-1.1.6.tar.gz
$ tar -zxvf canal.deployer-1.1.6.tar.gz
```

2) 配置 Canal

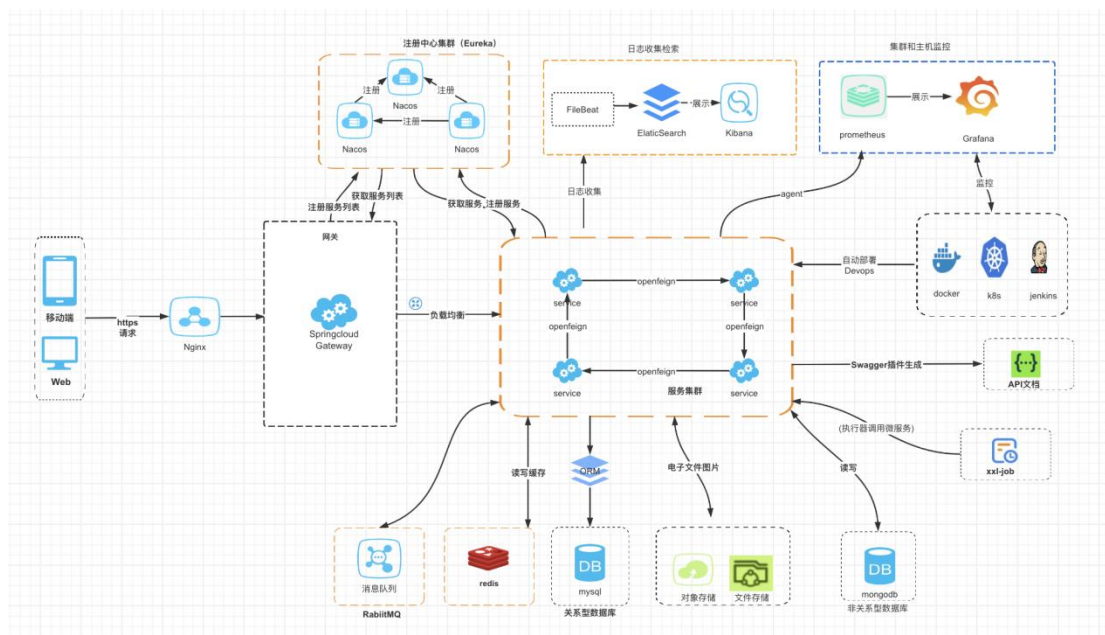
以下红色位置替换为实际使用的信息

需要先通过云数据库 rds 进入帐号管理创建 canal 需要使用的用户名（替换下文使用 **canalUser**）和密码（替换下文使用 **canalPass**）；**testdb** 是业务数据库使用（替换实际业务使用数据库）

```
$ vim /data/app/canal/conf/
$ mkdir /data/app/canal/conf/niux
$ vim /data/app/canal/conf/niux/instance.properties
# username/password
canal.instance.dbUsername=canalUser
canal.instance.dbPassword=canalPass
canal.instance.connectionCharset = UTF-8
canal.instance.defaultDatabaseName = testdb
# enable druid Decrypt database password
canal.instance.enableDruid=false
# table regex
canal.instance.filter.regex=canaldb\\.*
# table black regex
canal.instance.filter.black.regex=mysql\\.*slave_.*
```

3 安装部署

3.1 系统部署架构图



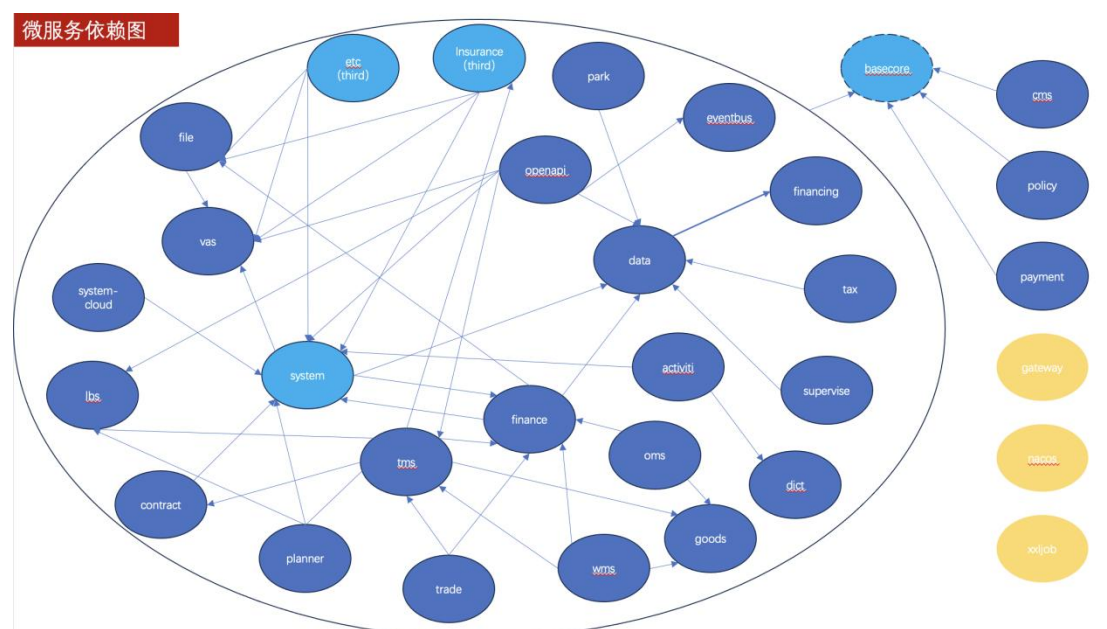
3.2 各应用简介

以实际代码提供中的微服务为准

微服务	微服务缩写	微服务应用名
流程引擎	activiti	niux-activiti
内容管理	cms	niux-cms
合同服务	contract	niux-contract
数据中心	data	niux-data
数据字典	dict	niux-dict
消息总线	eventbus	niux-eventbus
文件服务	file	niux-file
财务服务	finance	niux-finance
融资服务	financing	niux-financing
货物管理	goods	niux-goods
位置服务	lbs	niux-lbs
订单服务	oms	niux-oms
开放接口	openapi	niux-openapi
园区服务	park	niux-park
支付服务	payment	niux-payment
运筹规划	planner	niux-planner
策略规则	policy	niux-policy
交通监管	supervise	niux-supervise
系统服务	system	niux-system
税务服务	tax	niux-tax
第三方库	third	niux-third

运输管理	tms	niux-tms
电商交易	trade	niux-trade
增值服务	vas	niux-vas
仓储管理	wms	niux-wms
报表引擎	report	niux-report
nacos	nacos	niux-nacos
xxljob	xxljob	niux-xxljob
网关服务	gateway	niux-gateway

3.3 各应用间互访关系



3.4 脚本执行

3.4.1 脚本初始化

初始化脚本文件格式： niux-init.sql

- 1) 通过 Navicat for MySQL 登录连接 mysql 数据库。
- 2) 执行“ niux-init.sql ”脚本，将数据导入到 mysql 数据库中。

3.4.2 版本升级

以 4.3.0 版本脚本为例，脚本格式如下：

- 1) 4.3.0_DDL_序号_JIRA 号_问题描述_提交人_日期.sql
- 2) 4.3.0_DML_序号_JIRA 号_问题描述_提交人_日期.sql
- 3) 4.3.0_DSP_序号_JIRA 号_问题描述_提交人_日期.sql

3.4.2.1 首次升级

从“niux-init.sql”脚本执行，如版本从 4.2.8 版本开始

- 步骤 1、执行 4.2.8 文件夹中大于 4.2.8_DDL_12_... 的脚本。
- 步骤 2、执行 4.2.8 文件夹中大于 4.2.8_DML_24_... 的脚本。
- 步骤 3、执行 4.2.8 文件夹中大于 4.2.8_DSP_01_... 的脚本。
- 步骤 4、同理执行大于 4.2.8 文件夹的脚本，依次执行 DDL、DML、DSP 文件。
...以此类推，直到 4.3.0 的文件夹的脚本全部执行完。

3.4.2.2 日常升级

从 4.2.9 脚本升级到 4.3.0 版本

- 步骤 1、找到上次 4.2.9 已执行的 DDL、DML、DSP 脚本的序号。
《4.2.9_DDL_09_...》，《4.2.9_DML_05_...》，《4.2.9_DSP_03_...》。
- 步骤 2、执行 4.2.9 文件夹中大于 《4.2.9_DDL_09_...》的脚本。
- 步骤 3、执行 4.2.9 文件夹中大于 《4.2.9_DML_05_...》的脚本。
- 步骤 4、执行 4.2.9 文件夹中大于 《4.2.9_DSP_03_...》的脚本。
- 步骤 5、同理执行大于 4.2.9 文件夹的脚本，依次执行 DDL、DML、DSP 文件。
...以此类推，直到 4.3.0 的文件夹的脚本全部执行完。

3.5 jar 包部署

3.5.1 jar 启动脚本配置说明



restart-server-niu
x.sh

备注：脚本从商务处获取

- 1). 修改 restart-server-niuxsh 启动脚本的 nacos.abc.com:8848 为实际的 nacos 地址；如果 nacos 地址不对服务无法启动
- 2). 脚本中指定 jar 包上传的目录是/data/website_\$agent_code，本文档指定\$agent_code=test 为例，修改 test 为实际的目录名称

- 3). 脚本中 `spring.profiles.active` 指 `nacos` 配置命名空间（此处例如命名空间是 `test`）下 `niux-prod.yaml` 配置文件的这个 `prod`
- 4). 脚本中 `spring.cloud.nacos.config.namespace` 表示 `nacos` 配置文件的命名空间,本文档指定 `ab0dff2f-7df4-487f-927c-f743ab26daf0` 为例,最终修改为实际使用的命名空间 `id`
- 5). 脚本中 `spring.cloud.nacos.discovery.namespace` 表示 `nacos` 服务注册中心的命名空间,本文档指定 `ab0dff2f-7df4-487f-927c-f743ab26daf0` 为例,最终修改为实际使用的命名空间 `id`
- 6). 执行重启 `bash restart-server-niux.sh niux-gateway` 指重启的是 `/data/website_test/niux-gateway.jar` 的 `jar` 包
- 7). 批量重启
`bash restart-server-niux.sh all`

3.5.2 web 部署

首页代码部署目录 `/data/app/www/v2.0/niuxweb`

手机端代码部署目录 `/data/app/www/v2.0/niuxapp`

3.5.3 niux-gateway 部署

3.5.3.1 修改配置

通过登陆 `nacos` 的地址 <http://ip:8848> 修改 `niux-prod.yaml` 配置

3.5.3.2 服务重启

- 1) 启动
`$ bash restart-server-niux.sh niux-gateway`
- 2) 停止
`$ kill $(ps aux |grep niux-gateway.jar |grep nacos |awk '{print $2}')`
- 3) 重启
`$ bash restart-server-niux.sh niux-gateway`

3.5.4 niux-activiti 部署

web 服务：对外提供了 web 界面访问

jar 包名: `niux-activiti.jar`

3.5.4.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.4.2 服务重启

4) 启动

```
$ bash restart-server-niux.sh niux-activiti
```

5) 停止

```
$ kill $(ps aux |grep niux-activiti.jar |grep nacos |awk '{print $2}')
```

6) 重启

```
$ bash restart-server-niux.sh niux-activiti
```

3.5.5 cms 部署

cms 服务：内容管理

jar 包名: niux-cms.jar

3.5.5.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.5.2 服务重启

7) 启动

```
$ bash restart-server-niux.sh niux-cms
```

8) 停止

```
$ kill $(ps aux |grep niux-cms.jar |grep nacos |awk '{print $2}')
```

9) 重启

```
$ bash restart-server-niux.sh niux-cms
```

3.5.6 contract 部署

contract 服务：合同服务

jar 包名: niux-contract.jar

3.5.6.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.6.2 服务重启

10) 启动

```
$ bash restart-server-niux.sh niux-contract
```

11) 停止

```
$ kill $(ps aux |grep niux-cms.jar |grep nacos |awk '{print $2}')
```

12) 重启

```
$ bash restart-server-niux.sh niux-contract
```

3.5.7 data 部署

data 服务：数据中心。

jar 包名: niux-data.jar

3.5.7.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.7.2 服务重启

13) 启动

```
$ bash restart-server-niux.sh niux-data
```

14) 停止

```
$ kill $(ps aux |grep niux-data.jar |grep nacos |awk '{print $2}')
```

15) 重启

```
$ bash restart-server-niux.sh niux-data
```

3.5.8 dict 部署

dict : 数据字典

jar 包名: niux-dict.jar

3.5.8.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.8.2 服务重启

16) 启动

```
$ bash restart-server-niux.sh niux-dict
```

17) 停止

```
$ kill $(ps aux |grep niux-data.jar |grep nacos |awk '{print $2}')
```

18) 重启

```
$ bash restart-server-niux.sh niux-dict
```

3.5.9 eventbus 部署

eventbus: 消息总线

jar 包名: niux-eventbus.jar

3.5.9.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.9.2 服务重启

19) 启动

```
$ bash restart-server-niux.sh niux-eventbus
```

20) 停止

```
$ kill $(ps aux |grep niux-eventbus.jar |grep nacos |awk '{print $2}')
```

21) 重启

```
$ bash restart-server-niux.sh niux-eventbus
```

3.5.10 file 部署

file: 文件服务

jar 包名: niux-file.jar

3.5.10.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.10.2 服务重启

22) 启动

```
$ bash restart-server-niux.sh niux-file
```

23) 停止

```
$ kill $(ps aux |grep niux-file.jar |grep nacos |awk '{print $2}')
```

24) 重启

```
$ bash restart-server-niux.sh niux-file
```

3.5.11 finance 部署

finance: 财务服务

jar 包名: niux-finance.jar

3.5.11.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.11.2 服务重启

25) 启动

```
$ bash restart-server-niux.sh niux-finance
```

26) 停止

```
$ kill $(ps aux |grep niux-finance.jar |grep nacos |awk '{print $2}')
```

27) 重启

```
$ bash restart-server-niux.sh niux-finance
```

3.5.12 financing 部署

financing: 融资服务

jar 包名: niux-financing.jar

3.5.12.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.12.2 服务重启

28) 启动

```
$ bash restart-server-niux.sh niux-financing
```

29) 停止

```
$ kill $(ps aux |grep niux-financing.jar |grep nacos |awk '{print $2}')
```

30) 重启

```
$ bash restart-server-niux.sh niux-financing
```

3.5.13 goods 部署

goods: 货物管理

jar 包名: niux-goods.jar

3.5.13.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.13.2 服务重启

31) 启动

```
$ bash restart-server-niux.sh niux-goods
```

32) 停止

```
$ kill $(ps aux |grep niux-goods.jar |grep nacos |awk '{print $2}')
```

33) 重启

```
$ bash restart-server-niux.sh niux-goods
```

3.5.14 lbs 部署

lbs: 位置服务

jar 包名: niux-lbs.jar

3.5.14.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.14.2 服务重启

34) 启动

```
$ bash restart-server-niux.sh niux-lbs
```

35) 停止

```
$ kill $(ps aux |grep niux-lbs.jar |grep nacos |awk '{print $2}')
```

36) 重启

```
$ bash restart-server-niux.sh niux-lbs
```

3.5.15 openapi 部署

openapi: 开放接口

jar 包名: niux-openapi.jar

3.5.15.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.15.2 服务重启

37) 启动

```
$ bash restart-server-niux.sh niux-openapi
```

38) 停止

```
$ kill $(ps aux |grep niux-openapi.jar |grep nacos |awk '{print $2}')
```

39) 重启

```
$ bash restart-server-niux.sh niux-openapi
```

3.5.16 oms 部署

oms: 订单服务

jar 包名: niux-oms.jar

3.5.16.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.16.2 服务重启

40) 启动

```
$ bash restart-server-niux.sh niux-oms
```

41) 停止

```
$ kill $(ps aux |grep niux-oms.jar |grep nacos |awk '{print $2}')
```

42) 重启

```
$ bash restart-server-niux.sh niux-oms
```

3.5.17 park 部署

park: 园区服务

jar 包名: niux-park.jar

3.5.17.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.17.2 服务重启

43) 启动

```
$ bash restart-server-niux.sh niux-park
```

44) 停止

```
$ kill $(ps aux |grep niux-park.jar |grep nacos |awk '{print $2}')
```

45) 重启

```
$ bash restart-server-niux.sh niux-park
```

3.5.18 payment 部署

payment: 支付服务

jar 包名: niux-payment.jar

3.5.18.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.18.2 服务重启

46) 启动

```
$ bash restart-server-niux.sh niux-payment
```

47) 停止

```
$ kill $(ps aux |grep niux-payment.jar |grep nacos |awk '{print $2}')
```

48) 重启

```
$ bash restart-server-niux.sh niux-payment
```

3.5.19 planner 部署

planner: 运筹规划

jar 包名: niux-planner.jar

3.5.19.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.19.2 服务重启

49) 启动

```
$ bash restart-server-niux.sh niux-planner
```

50) 停止

```
$ kill $(ps aux |grep -i niux-planner.jar |grep nacos |awk '{print $2}')
```

51) 重启

```
$ bash restart-server-niux.sh niux-planner
```

3.5.20 policy 部署

policy: 运筹规划

jar 包名: niux-policy.jar

3.5.20.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.20.2 服务重启

52) 启动

```
$ bash restart-server-niux.sh niux-policy
```

53) 停止

```
$ kill $(ps aux |grep niux-policy.jar |grep nacos |awk '{print $2}')
```

54) 重启

```
$ bash restart-server-niux.sh niux-policy
```

3.5.21 supervise 部署

supervise: 交通监管

jar 包名: niux-supervise.jar

3.5.21.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.21.2 服务重启

55) 启动

```
$ bash restart-server-niux.sh niux-supervise
```

56) 停止

```
$ kill $(ps aux |grep niux-supervise.jar |grep nacos |awk '{print $2}')
```

57) 重启

```
$ bash restart-server-niux.sh niux-supervise
```

3.5.22 system 部署

system: 系统服务

jar 包名: niux-system.jar

3.5.22.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.22.2 服务重启

58) 启动

```
$ bash restart-server-niux.sh niux-system
```

59) 停止

```
$ kill $(ps aux |grep niux-system.jar |grep nacos |awk '{print $2}')
```

60) 重启

```
$ bash restart-server-niux.sh niux-system
```

3.5.23 tax 部署

tax: 税务服务

jar 包名: niux-tax.jar

3.5.23.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.23.2 服务重启

61) 启动

```
$ bash restart-server-niux.sh niux-tax
```

62) 停止

```
$ kill $(ps aux |grep niux-tax.jar |grep nacos |awk '{print $2}')
```

63) 重启

```
$ bash restart-server-niux.sh niux-tax
```

3.5.24 asset 部署

jar 包名: niux-asset.jar

3.5.24.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.24.2 服务重启

64) 启动

```
$ bash restart-server-niux.sh niux-asset
```

65) 停止

```
$ kill $(ps aux |grep niux-asset.jar |grep nacos |awk '{print $2}')
```

66) 重启

```
$ bash restart-server-niux.sh niux-asset
```

3.5.25 tms 部署

tms: 运输管理

jar 包名: niux-tms.jar

3.5.25.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.25.2 服务重启

67) 启动

```
$ bash restart-server-niux.sh niux-tms
```

68) 停止

```
$ kill $(ps aux |grep niux-tms.jar |grep nacos |awk '{print $2}')
```

69) 重启

```
$ bash restart-server-niux.sh niux-tms
```

3.5.26 trade 部署

trade: 电商交易

jar 包名: niux-trade.jar

3.5.26.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.26.2 服务重启

70) 启动

```
$ bash restart-server-niux.sh niux-trade
```

71) 停止

```
$ kill $(ps aux |grep niux-trade.jar |grep nacos |awk '{print $2}')
```

72) 重启

```
$ bash restart-server-niux.sh niux-trade
```

3.5.27 vas 部署

vas: 增值服务

jar 包名: niux-vas.jar

3.5.27.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.27.2 服务重启

73) 启动

```
$ bash restart-server-niux.sh niux-vas
```

74) 停止

```
$ kill $(ps aux |grep niux-vas.jar |grep nacos |awk '{print $2}')
```

75) 重启

```
$ bash restart-server-niux.sh niux-vas
```

3.5.28 wms 部署

wms: 仓储管理

jar 包名: niux-wms.jar

3.5.28.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.28.2 服务重启

76) 启动

```
$ bash restart-server-niux.sh niux-wms
```

77) 停止

```
$ kill $(ps aux |grep niux-wms.jar |grep nacos |awk '{print $2}')
```

78) 重启

```
$ bash restart-server-niux.sh niux-wms
```

3.5.29 report 部署

report: 报表引擎

jar 包名: niux-report.jar

3.5.29.1 修改配置

通过登陆 nacos 的地址 <http://ip:8848> 修改 niux-prod.yaml 配置

3.5.29.2 服务重启

79) 启动

```
$ bash restart-server-niux.sh niux-report
```

80) 停止

```
$ kill $(ps aux |grep niux-report.jar |grep nacos |awk '{print $2}')
```

81) 重启

```
$ bash restart-server-niux.sh niux-report
```

3.5.30 检查进程中未启动的 java 服务

```
$ cd /data/website_test
$ agent_code=test; ll *jar|egrep -v "($(ps aux |sed -n
"/$agent_code/s/\(.*\)website_$agent_code\\/ \(.*\)\\.jar\\(.*\)\\/2\\.jar/gp" |xargs
-n 100|sed -n "s/ /|/gp"))"
```

3.5.31 检查重复的 java 服务

```
$ cd /data/website_test
$ agent_code=test; ps aux |grep -i java |grep -i $agent_code |sed -n
"s/\(.*\)\\/ \(.*\)\\.jar\\(.*\)\\/2/gp" |sort |uniq -c |sort -nr
```

3.6 登录账号

部署好系统后，在浏览器打开连接 <http://192.168.0.1>，输入对应的账号和密码进行登录。默认账号 12800000000 和默认密码 123456。

4 小程序部署

4.1 申请账号

参考官方文档

<https://developers.weixin.qq.com/miniprogram/dev/quickstart/basic/getstart.html>#申请帐号

申请后的 appID 如图。



4.2 安装开发者工具

参考官方文档

<https://developers.weixin.qq.com/miniprogram/dev/quickstart/basic/getstart.html#安装开发工具>

4.3 构建程序

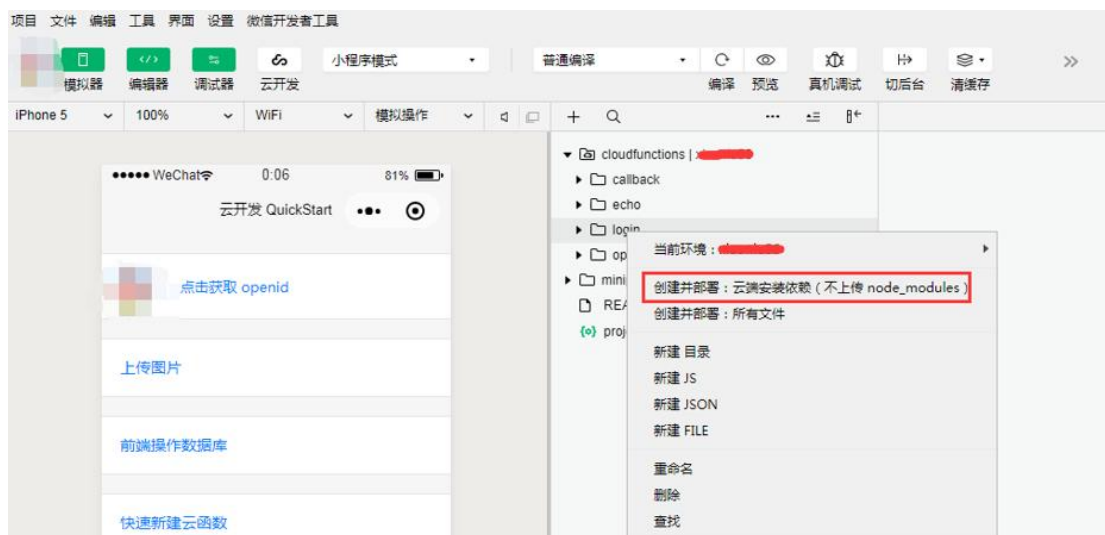
打开“微信开发者工具”新增小程序



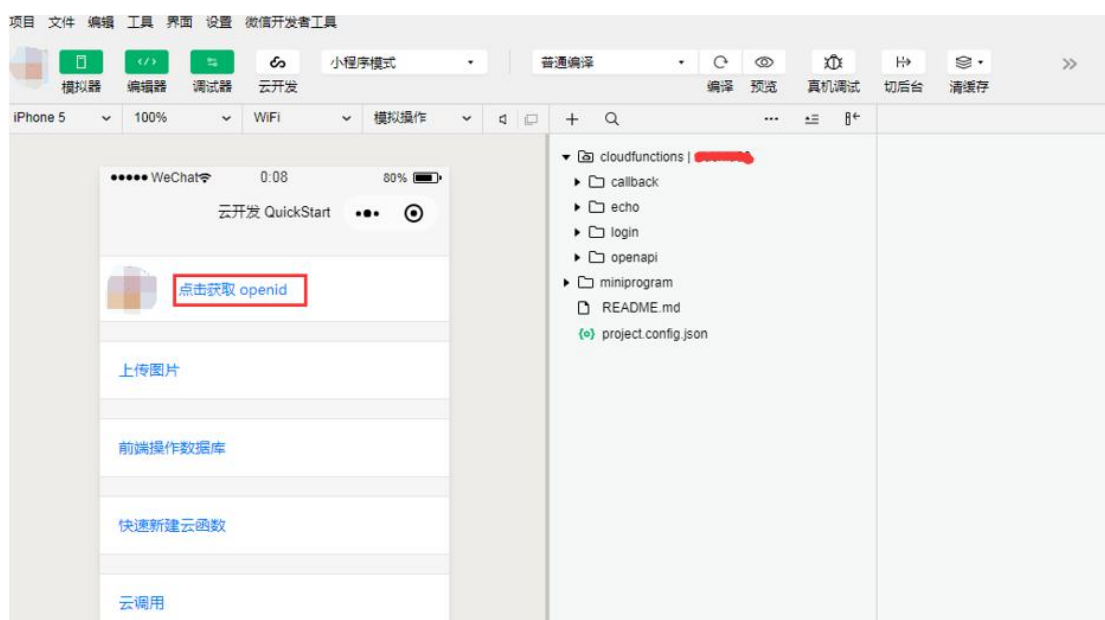
填写项目信息



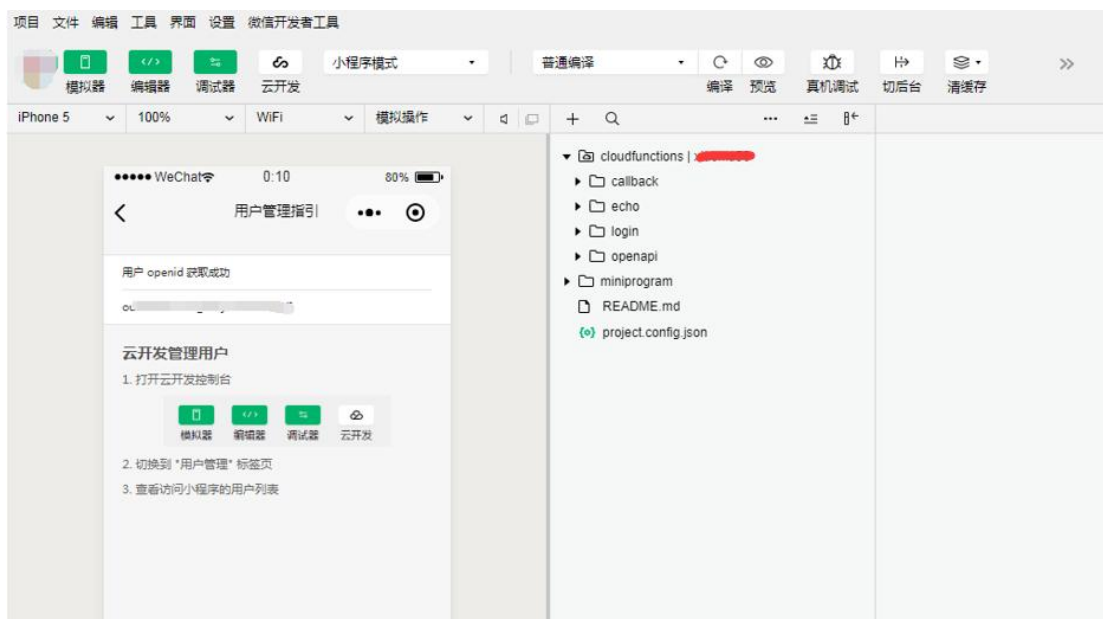
创建并部署



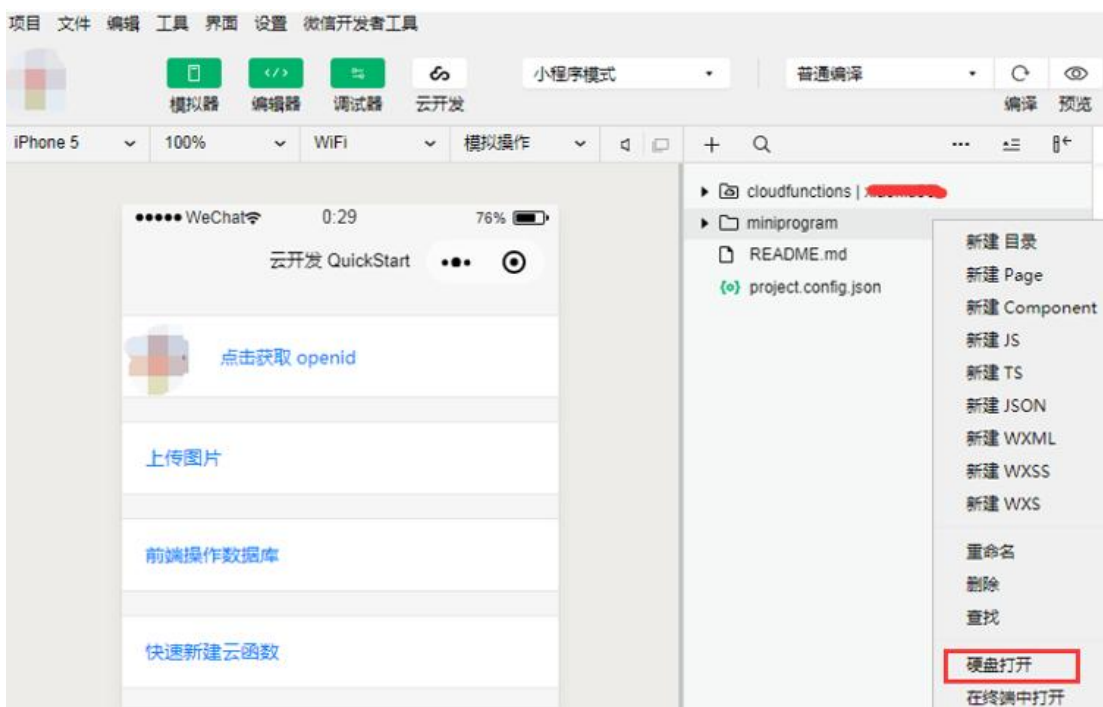
获取 openid



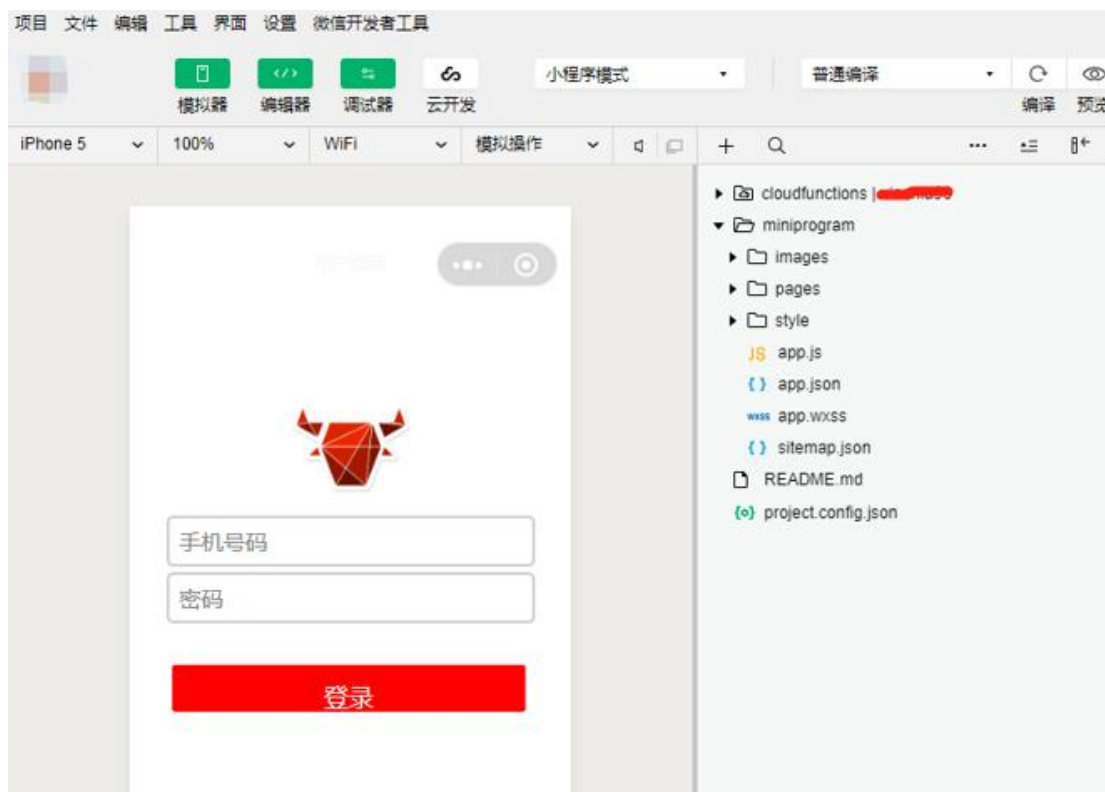
获取成功后如下图



替换项目代码，右键“miniprogram”打开目录，用 xiaoniu-miniprogram-x.y.z 文件夹下的 miniprogram 替换。



替换后如下



4.4 修改配置文件

1) 打开 miniprogram/config.js

```
let onlineSetting = {
  #app 名称
  appName: "运输圈",
  # appSecret: 查看 xn_m_app.private_key
  appSecret: "private_key",
  # appKey: 查看 xn_m_app.appkey
  appKey: "appkey",
  releaseUrl: 'https://www.abc.com',
  # 登录页 Logo 图片名称
  imgName: 'xiaoniu_w@2x.png',
  # 版本号
  versionCode: '版本 4.2.6',
  # 找油网 AppId
  oilAppId: 'AppId',
  # 找油网 AppAey
  oilAppAey: 'AppAey',
  # 找油网域名地址
  oilUrl: 'https://test.lvjy-api.51zhaoyou.com/h5/laolv/common/access',
  # 客户电话
  telephoneNumber: '4000062587',
```

```
# 百度OCR 申请id
client_id: "client_id",
# 百度OCR 申请secret
client_secret: "client_secret",
# 高德地图 key
regeo_mapWX: "key"
}
```

2) 小程序的图片路径

将小程序 `appimg` 目录拷贝到服务器 `/data/app/www` 下，以便小程序调用。

4.5 发布上线

参考官方文档

<https://developers.weixin.qq.com/miniprogram/dev/framework/quickstart/release.html#发布上线>

联系我们



电话：400-006-2587

邮件：service@niuinfo.com

网站：http://www.niuinfo.com

地址：上海市嘉定区银翔路 609 号 3 层 (201802)